FACULTY OF ENGINEERING AND ARCHITECTURE

# FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

Simon Coudenys
Student number: 02101804

Supervisor: prof. dr. ir. Peter Ossieur, ir. Gertjan Coudyzer
Counsellor: Jonas De Busscher, Axl Bomhals, Dr. Jakob Declercq

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics and ICT Engineering Technology - Embedded Systems

GHENT UNIVERSITY

Department of Information Technology
Chair: prof. dr. ir. Peter Ossieur
Faculty of Engineering and Architecture
Academic year 2025-2026

*This page is intentionally left blank.*

*This page is intentionally left blank.*

FACULTY OF ENGINEERING AND ARCHITECTURE

# FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

Simon Coudenys
Student number: 02101804

Supervisor: prof. dr. ir. Peter Ossieur, ir. Gertjan Coudyzer
Counsellor: Jonas De Busscher, Axl Bomhals, Dr. Jakob Declercq

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics and ICT Engineering Technology - Embedded Systems

GHENT UNIVERSITY

Department of Information Technology
Chair: prof. dr. ir. Peter Ossieur
Faculty of Engineering and Architecture
Academic year 2025-2026

*This page is intentionally left blank.*

# Acknowledgements

Laboris magna irure quis sit et veniam voluptate duis elit esse do excepteur. Officia laboris adipisicing laboris duis ut cillum aliquip. Do veniam aute non do cillum sit pariatur consectetur adipisicing qui. Lorem Lorem veniam minim exercitation cupidatat tempor do aute dolore. Nostrud adipisicing tempor quis qui fugiat sint quis ut aute eiusmod Lorem enim veniam. Elit labore Lorem do proident amet anim exercitation culpa enim cillum qui. Aliquip reprehenderit commodo labore minim laborum mollit ut veniam in irure. Minim sit in non aliqua ut elit veniam irure pariatur non velit consequat. Cupidatat veniam occaecat culpa eu irure aute consectetur dolore culpa deserunt quis officia do. Aliqua aliqua eu velit excepteur culpa ea dolore occaecat. Ea est cillum nulla ea cupidatat esse adipisicing ut. Cillum amet esse ex ex enim elit excepteur ullamco in qui aliqua Lorem veniam. Sunt consectetur aliquip commodo exercitation pariatur id tempor adipisicing aliqua. Fugiat cillum ea irure laborum consectetur ipsum. Excepteur laboris eu veniam ex ad consequat esse id ullamco ea cupidatat ad nisi. Aute nisi velit excepteur adipisicing dolore pariatur sunt. Dolore mollit cupidatat esse sint ipsum est nulla nisi eu cupidatat excepteur sint cillum. Culpa irure magna culpa eiusmod ut nisi ipsum veniam elit est occaecat veniam. Sunt adipisicing eu minim qui ea laboris ullamco ut. Aliquip occaecat in occaecat ut tempor dolore sunt consequat qui cillum reprehenderit. Voluptate commodo cupidatat ullamco sint aliquip veniam consequat do. Nisi velit excepteur aute laboris aliqua fugiat minim occaecat quis non incididunt. Lorem velit laboris excepteur minim aute laborum ea occaecat duis. Est proident dolor culpa in. Ex sit sint voluptate esse fugiat laborum aliquip laboris quis irure fugiat eiusmod. Dolor occaecat deserunt quis occaecat anim ullamco occaecat dolor. Ea duis eiusmod veniam in fugiat aute dolore officia sint proident deserunt irure est non. Id nulla velit velit aliqua duis dolor Lorem. Sit labore qui dolor occaecat sit irure eiusmod fugiat. In aliqua nisi est in. Tempor aliqua labore magna fugiat incididunt sunt minim sit ut dolor consequat. Cupidatat ex ut nostrud sint officia cillum consequat proident aliquip voluptate. Aliqua reprehenderit dolore eiusmod non. Esse eu exercitation laboris aliqua enim magna sunt id veniam anim. Laboris in nostrud aliqua anim nostrud adipisicing sit velit id eiusmod commodo Lorem deserunt commodo. Eiusmod commodo culpa ad tempor. Excepteur sint fugiat ex veniam esse fugiat laboris mollit. Exercitation id ex culpa aliquip in magna quis consequat. Qui aliquip consequat amet ad laboris non tempor minim eiusmod officia aliquip ullamco. Anim adipisicing magna do et dolore. Quis non aliquip eiusmod cupidatat. Excepteur adipisicing minim cupidatat adipisicing irure. Sunt proident fugiat nulla ea minim esse non. Exercitation dolor non nulla est et adipisicing est dolore pariatur reprehenderit cillum eiusmod. Eu sunt aliquip quis ea nulla ea veniam quis consequat. Veniam proident id eu cillum enim sit enim id sunt. Laboris magna irure quis sit et veniam voluptate duis elit esse do excepteur. Officia laboris adipisicing laboris duis ut cillum aliquip. Do veniam aute non do cillum sit pariatur consectetur adipisicing qui. Lorem Lorem veniam minim exercitation cupidatat tempor do aute dolore. Nostrud adipisicing tempor quis qui fugiat sint quis ut aute eiusmod Lorem enim veniam. Elit labore Lorem do proident amet anim exercitation culpa enim cillum qui. Aliquip reprehenderit commodo labore minim laborum mollit ut veniam in irure. Minim sit in non aliqua ut elit veniam irure pariatur non velit consequat. Cupidatat veniam occaecat culpa eu irure aute consectetur dolore culpa deserunt quis officia do. Aliqua aliqua eu velit excepteur culpa ea dolore occaecat. Ea est cillum nulla ea cupidatat esse adipisicing ut. Cillum amet esse ex ex enim elit excepteur ullamco in qui aliqua Lorem veniam. Sunt consectetur aliquip commodo exercitation pariatur id tempor adipisicing aliqua. Fugiat cillum ea irure laborum consectetur ipsum. Excepteur laboris eu veniam ex ad consequat esse id ullamco ea cupidatat ad nisi. Aute nisi velit excepteur adipisicing dolore pariatur sunt. Dolore mollit cupidatat esse sint ipsum est nulla nisi eu cupidatat excepteur sint cillum. Culpa irure magna culpa eiusmod ut nisi ipsum veniam elit est occaecat veniam. Sunt adipisicing eu minim qui ea laboris ullamco ut. Aliquip occaecat in occaecat ut tempor dolore sunt consequat qui cillum reprehenderit. Voluptate commodo cupidatat ullamco sint aliquip veniam consequat do. Nisi velit excepteur aute laboris aliqua fugiat minim occaecat quis non incididunt. Lorem velit laboris excepteur minim aute laborum ea occaecat duis. Est proident dolor culpa in. Ex sit sint voluptate esse fugiat laborum aliquip laboris quis irure fugiat eiusmod. Dolor occaecat deserunt quis occaecat anim ullamco occaecat dolor. Ea duis eiusmod veniam in fugiat aute dolore officia sint proident deserunt irure est non. Id nulla velit velit aliqua duis dolor Lorem. Sit labore qui dolor occaecat sit irure eiusmod fugiat. In aliqua nisi est in. Tempor aliqua labore magna fugiat

incididunt sunt minim sit ut dolor consequat. Cupidatat ex ut nostrud sint officia cillum consequat proident aliquip voluptate. Aliqua reprehenderit dolore eiusmod non. Esse eu exercitation laboris aliqua enim magna sunt id veniam anim. Laboris in nostrud aliqua anim nostrud adipisicing sit velit id eiusmod commodo Lorem deserunt commodo. Eiusmod commodo culpa ad tempor. Excepteur sint fugiat ex veniam esse fugiat laboris mollit. Exercitation id ex culpa aliquip in magna quis consequat. Qui aliquip consequat amet ad laboris non tempor minim eiusmod officia aliquip ullamco. Anim adipisicing magna do et dolore. Quis non aliquip eiusmod cupidatat. Excepteur adipisicing minim cupidatat adipisicing irure. Sunt proident fugiat nulla ea minim esse non. Exercitation dolor non nulla est et adipisicing est dolore pariatur reprehenderit cillum eiusmod. Eu sunt aliquip quis ea nulla ea veniam quis consequat. Veniam proident id eu cillum enim sit enim id sunt.

# Admission to loan

"The author(s) gives (give) permission to make this master's dissertation available for consultation and to copy parts of this master's dissertation for personal use. In the case of any other use, the copyright terms have to be respected, in particular with regard to the obligation to state expressly the source when quoting results from this master's dissertation."

# Toegang tot bruikleen

"De auteur(s) geeft (geven) de toelating dit afstudeerwerk voor consultatie beschikbaar te stellen en delen van het afstudeerwerk te kopiëren voor persoonlijk gebruik. Elk ander gebruik valt onder de beperkingen van het auteursrecht, in het bijzonder met betrekking tot de verplichting de bron uitdrukkelijk te vermelden bij het aanhalen van resultaten uit dit afstudeerwerk."

Simon Coudenys, Mei 2026

*This page is intentionally left blank.*

# Thesis as part of an exam

"This master's dissertation is part of an exam. Any comments formulated by the assessment committee during the oral presentation of the master's dissertation are not included in this text."

Simon Coudenys, Mei 2026

*This page is intentionally left blank.*

# FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

by
Simon Coudenys

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics and ICT Engineering Technology - Embedded Systems

Supervisor: prof. dr. ir. Peter Ossieur, ir. Gertjan Coudyzer
Counsellor: Jonas De Busscher, Axl Bomhals, Dr. Jakob Declercq

Faculty of Engineering and Architecture
Ghent University

Department of Information Technology
Chair: prof. dr. ir. Peter Ossieur

**Abstract** - Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Nibh sit amet commodo nulla facilisi nullam. Amet venenatis urna cursus eget nunc scelerisque. Mauris cursus mattis molestie a iaculis at. Est sit amet facilisis magna etiam tempor orci eu. Malesuada nunc vel risus commodo viverra maecenas accumsan. Volutpat diam ut venenatis tellus in metus vulputate. Adipiscing tristique risus nec feugiat in. Orci sagittis eu volutpat odio facilisis mauris sit. Posuere ac ut consequat semper. Amet commodo nulla facilisi nullam vehicula ipsum a arcu cursus. Odio euismod lacinia at quis risus. Natoque penatibus et magnis dis parturient. Eleifend donec pretium vulputate sapien nec. Odio pellentesque diam volutpat commodo sed. Tellus molestie nunc non blandit massa enim nec dui nunc. Turpis egestas integer eget aliquet. Consequat id porta nibh venenatis cras sed. Purus in mollis nunc sed id semper risus in hendrerit. Pellentesque eu tincidunt tortor aliquam nulla facilisi cras fermentum. Porttitor eget dolor morbi non. Vivamus at augue eget arcu dictum. Eget felis eget nunc lobortis. Urna nunc id cursus metus aliquam eleifend mi. Est pellentesque elit ullamcorper dignissim cras. A erat nam at lectus.

**Keywords** - DSP, Baudrate sampling, 16QAM, Intra datacenter, Coherent optical links

*This page is intentionally left blank.*

# Guidelines for the PhD Symposium Papers

Joris Thybaut

Supervisor(s): Eric Laermans, Luc Dupré

***Abstract*— This article explains how to format your article for the Proceedings of the FEA PhD Symposium using Microsoft Word as a word processor.**

***Keywords*— layout, MS Word, FEA PhD Symposium**

## I. Introduction

This sample file shows what the layout of your paper should be. The styles of this file have been created starting from Word's *NORMAL.DOT* template. This sample file should work for all recent MS Office versions. A polemic on the use of MS Word versus LaTeX as a word processor would probably be never ending [1][2].

We recommend a *double column* style as this makes the article easier to read. The column width is 88,5mm. In this sample file you will find examples for the layout of displayed equations, theorems, tables, figures, etc.

The author of this sample document is an advanced MS Word user, but certainly not an expert. Any comments are welcome…

## II. Getting started

### A. Creating A New Paper based on a MS Word template file

The proper use of this template requires that one copies it into the template directory (something like this: C:\Documents and Settings\USERNAME\Application Data\ Microsoft\Templates) and that, when opening a new document, one creates a new document from this template. This sample document shows what various styles and elements (tables, figures,…) in this PhD Symposium style look like.

### B. 'Help, I seem unable to use MS Word template files'

A probably more easy way to generate a new Microsoft Word document following the suggested guide lines of the Symposium Proceedings, if you fail to follow the proper way, is edit this sample file and to rename it into a .doc file… The easiest way, in the most recent MS Word versions, is to double-click on the template file saved on, e.g., the desktop.

## III. PhD Symposium Paper styles

### A. Style Reference

Here's an alphabetical reference for the styles used in the FEA PhD Symposium papers. Some are borrowed from Word's *NORMAL.DOT*; others are specific to the Symposium style. Note that paragraphs do not require special styling and should be left with the "Normal" style (10pt Times New Roman).

**Abstract**— Includes all text for your abstract. Formatting is 9-point bold. (The word "abstract" itself should be italic instead of bold)

**Authors**— Comprises the entire comma-delimited list of authors, starting with the principal author, i.e., the presenting PhD student.

**AuthorInfo**— This style is meant for author affiliation information. This style should be unique within the paper. It produces a special footnote on the first page. The footnote has a non-printing symbol as its reference marker so as not to interfere with the numbering of regular footnotes later in the paper. [1]

**Heading 1—** Top-level heading. Auto-numbered, so you shouldn't add Roman numerals of your own. Appears centered on the column with smallcaps.

**Heading 2—** Level-two heading. Auto-numbered, so you shouldn't add capital letters of your own.

**Heading 3—** Level-three heading. Auto-numbered, so you shouldn't add Arabic numerals of your own. Note that only heading levels 1 through 3 are supported in the Symposium style. (Deeper subsectioning levels are not recommended and probably totally useless in a two pages long paper.)

**KeyWords—** Optional list of keywords. If you supply no index terms, this style should be removed entirely.

**Lemma—** Heading (only) for a lemma.

**List Number—** Regular, numbered, and bulleted lists. Used just as within Word normally. Each entire list receives the list style, and carriage returns within the list area define list items.

**References—** Reference section (bibliography). A special kind of numbered list. There should be only one *References* style area in your paper. Auto-numbered, so you shouldn't add numbering of your own.

**Theorem**— Heading (only) for a theorem. Derived from *Heading 3*.

**Title—** Full paper title. Should be the first item in the paper. Appears in 24-point Times New Roman type centered on the page.

### B. Order of Appearance

Here's a list of styles specific to the Symposium layout in order of appearance:

**Title**
**Authors**
**AuthorInfo** (special footnote)
**Abstract**
**IndexTerms**
**Heading 1**
**Heading 2**

J. Thybaut is with the Chemical Engineering Department, Ghent University (UGent), Gent, Belgium. E-mail: Joris.Thybaut@UGent.be .

[1] This is a regular, numbered footnote.

**Heading 3**
**References**
All other styles may be repeated as often as necessary.

## IV. Math

The incorporation of mathematical formulas in the text is probably one of the major issues discussed when comparing MS Word to LaTeX. LaTeX is indeed famous for its equation editing possibilities, while Microsoft's ordinary Equation Editor serves the needs for a limited number of relatively simple equations but becomes cumbersome when used in texts with lots of extensive equations.

In the Symposium Proceedings style the justification of equations should be centred and the numbering should be right-justified. By pressing <TAB> once, one will reach the centered tabulator stop and can enter the equation. After inserting another <TAB> the right-justified position will be reached and the equation number can be inserted. As concerns the use of fonts (bold, italic,…), we refer to the common guidelines for any scientific work [3].

Here is an example of an equation within a theorem:

*Theorem 1 (Theorem name) Consider the system*

$$\begin{aligned} \dot{x} &= A.x + B.u \\ y &= C.x + D.u \end{aligned} \tag{1}$$

*If* A *is stable, then the pair {*A*,* B*} is stabilizable. Moreover this holds for any* B.

An automatic equation numbering can be obtained by the use of the appropriate field. To insert an equation number one should go to *insert, field* and select '*Seq*' in '*Field names'*. After pressing the button '*Field codes'* one can enter, e.g., '*Equation'*, for the automatic numbering of the equations. One can also simply execute the macro 'equation numbering'.

## V. Markup

Markup defines the parts of your paper: the title, headings, lists, and so forth. Each of these elements is given an appropriate name, and certain formatting instructions are associated with that name. In Word, the mechanism for associating an element name with its formatting conventions is called a style.

Style names appear in the pull down style menu along the left hand side of your Word window. In the more recent version you can let appear a style window with extra possibilities on the right hand side of the screen. Typically, you highlight an area of text that you wished to designate with a certain style, then select the appropriate name on the style menu.

## VI. Tables and Figures

Tables and figures are centre-justified, which can be obtained by pressing <TAB> once. Captions for tables should be defined before the table item itself and should be inserted in a proper way, i.e., via '*Insert'*, '*Reference'*, '*Caption'* and then select '*Table*'. Captions to figures are inserted in an analogous way, but should be put after the figure.

Table 1 The Caption comes Before the Table

| | title page | odd page | even page |
|---|---|---|---|
| onesided | leftTEXT | leftTEXT | leftTEXT |
| twosided | leftTEXT | rightTEXT | leftTEXT |

Figure 1 The caption comes after the figure.

## VII. Referencing

Another LaTeX versus MS Word issue is referencing. Many MS Word users are unaware how to do this or experience problems in doing so. Referencing is only possible when (hidden) bookmarks have been inserted.

### A. Hidden bookmarks

When inserting a Table or Figure caption, or when inserting a numbered list (such as the references), hidden bookmarks are created automatically. One can refer to these hidden bookmarks via '*Insert'*, '*Reference'*, '*Cross-Reference'* and than select the '*reference type'*, select what should be included (most probably only the number) and then select the actual item you want to refer to. These are references to Figure 1 and Table 1 and to reference [1].

Unexpected things can happen when the hidden bookmark is moved away from its original location. This can, e.g., happen when inserting a reference Y in between two references X and Z. To properly do so one should put the cursor at the end of reference X and press <ENTER>. This is the only way the hidden bookmark of reference Z will stay with that reference and references to this bookmark will refer appropriately. Suppose one hits <ENTER> at the beginning of reference Z the hidden bookmark will not move with the reference and references to this bookmark will point to the newly inserted reference Y instead to reference Z.

When inserting a new item in between two others references are not updated immediately. They are only update when opening the document, printing it or looking at the print preview.

### B. User inserted bookmarks

When inserting equation numbers no (hidden) bookmarks are inserted automatically. To be able to refer to an equation number, one has to manually create a bookmark containing that equation number. After selecting the equation number the bookmark can be inserted via '*Insert'*, '*Bookmark'* and then type a name for this bookmark. Once this has been done, a cross-reference to this bookmark can be inserted following the procedure described above. This is a reference to equation (1).


## Acknowledgements

The authors would like to acknowledge the suggestions of many people.


## References


[1] Leslie Lamport, *A Document Preparation System: LaTeX, User's Guide and Reference Manual*, Addison Wesley Publishing Company, 1986.

[2] Helmut Kopka, *LaTeX, eine einführung*, Addison-Wesley, 1989.

[3] E. R. Cohen and P. Giacomo, *Symbols, Units, Nomenclature and Fundamental Constants in Physics*, Document I.U.P.A.P.-25 (SUNAMCO 87-1), 1987

*This page is intentionally left blank.*

# Table of Contents

# List of Figures



# List of Tables

*This page is intentionally left blank.*

# List of Acronyms

**IMDD** intensity modulation and direct detection

**PAM4** pulse amplitude modulation 4-level

**QPSK** quatrature phase shift keying

**QAM** quadrature amplitude modulation

**16QAM** 16-state quadrature amplitude modulation

**ASIC** application specific integrated circuit

**DSP** digital dignal processing

**BPS** blind phase search

**SBPS** serialised binary phase search

**SE** spread estimation

**PU** phase unwrapper

**PC** phase compensator

**FPGA** field programmable gate array

**CORDIC** coordinate rotation digital computer

**IP** intellectual property

**VHDL** VHSIC Hardware Description Language

**SNR** signal-to-noise ratio

**LUT** look-up table

**CFO** carrier frequency offset

**VHSIC** very high speed integrated circuit

**LO** local oscillator

**ASE** amplified spontaneous emission

**FFT** fast fourier-transform

**MUX** multiplexer

*This page is intentionally left blank.*

# 1 Introduction

## 1.1 Context and motivation

The rapid growth of cloud computing has led to a strong increase in the amount of data exchanged inside modern datacenters. Services such as video and music streaming, cloud storage and online gaming already require large amounts of internal network bandwidth . More recently, the emergence of AI services such as ChatGPT, Gemini and Claude has accelerated this trend even further . These applications rely on large clusters of GPUs and servers that continually exchange data. As a result, the communication links between servers, switches and racks inside a datacenter are becoming an increasingly important bottleneck for these type of services.

insert ref of 80% of internet communication happens on intra-datacenter links

insert ref

To support this growth, datacenter interconnects are evolving towards 800 Gbit/s and 1.6 Tbit/s optical transceivers. These links are expected to play a key role in next-generation AI datacenters, where communication between accelerators and switching equipment must remain both high-speed and energy-efficient [1].

Today, most short-reach intra-datacenter optical links rely on intensity modulation and direct detection (IMDD), often using pulse amplitude modulation 4-level (PAM4) signalling. These techniques are relatively simple and power-efficient, which makes them attractive for short optical links. However, as symbol rates and transmission distances continue to increase, IMDD systems are reaching their limits in terms of chromatic dispersion tolerance, achievable spectral efficiency and receiver sensitivity.

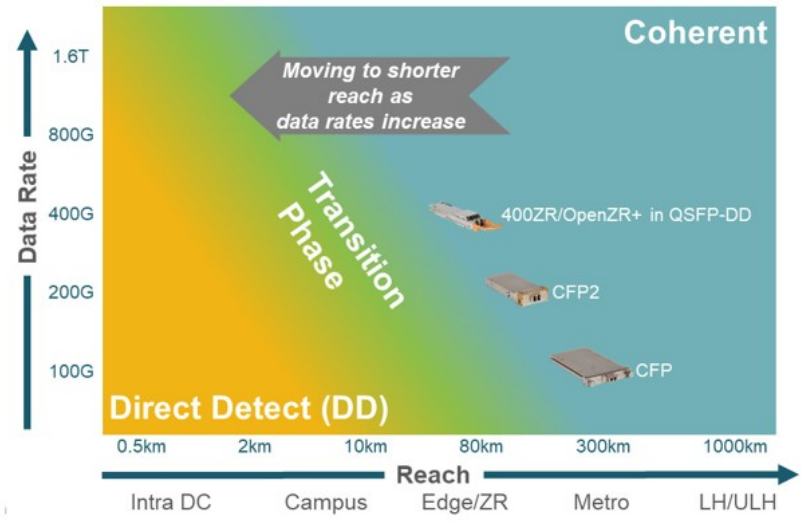



Figure 1.1: This image needs to be referenced still.

make more clear why this picture should be here

Coherent optical communication offers an attractive alternative because it enables the use of higher-order modulation formats such as 16-state quadrature amplitude modulation (16QAM) while providing

improved sensitivity and better tolerance to fiber impairments. Experimental comparisons between coherent and IMDD transceivers for intra-datacenter links have shown that coherent solutions can provide a larger optical power budget while maintaining comparable application specific integrated circuit (ASIC) power consumption [2].

## 1.2 Problem statement

Although coherent optical communication provides important advantages in terms of spectral efficiency and receiver sensitivity, these benefits come at the cost of increased receiver complexity. In particular, the digital dignal processing (DSP) chain of a coherent receiver contains computationally expensive blocks for frequency recovery and carrier phase recovery.

Carrier recovery is responsible for compensating the residual frequency offset and phase noise caused by the finite linewidth of the transmitter and local oscillator lasers. In conventional coherent receivers, this is often achieved through a combination of fourth-power frequency estimation and blind phase search (BPS). The frequency recovery stage performs a coarse estimation of the carrier frequency offset, after which the phase recovery stage compensates the remaining phase rotation and phase noise.

However, both of these techniques become increasingly expensive as baudrates increase. Fourth-power frequency estimation typically relies on large FFT sizes in order to obtain sufficient frequency resolution. At the same time, blind phase search requires many test angles to achieve accurate phase estimation. This results in a significant amount of complex multiplications and memory usage, making carrier recovery one of the dominant contributors to the overall DSP complexity of a coherent receiver. source if applicable

These complexity challenges are particularly important in intra-datacenter links, where power consumption and latency are critical constraints. In such environments, it is not sufficient for a carrier recovery algorithm to be accurate alone; it must also be implementable efficiently in hardware.

This thesis therefore investigates a low-complexity field programmable gate array (FPGA)-based carrier recovery architecture for baudrate-sampled 16QAM coherent intra-datacenter links. The proposed approach aims to reduce the computational complexity of blind phase search while extending its effective frequency tracking range. This is achieved by combining a serialized blind phase search algorithm with an additional phase spread estimation block, allowing more robust carrier recovery with lower overall hardware cost.

## 1.3 Objectives of this thesis

A mathematical model of the proposed low-complexity carrier phase recovery system has already been developed in Simulink and will be discussed in more detail in Chapter 3. In this thesis, the implementation of this phase recovery architecture on an FPGA will be presented in detail.

The proposed architecture forms the final stage of the receiver DSP chain, directly before the symbol decision stage.

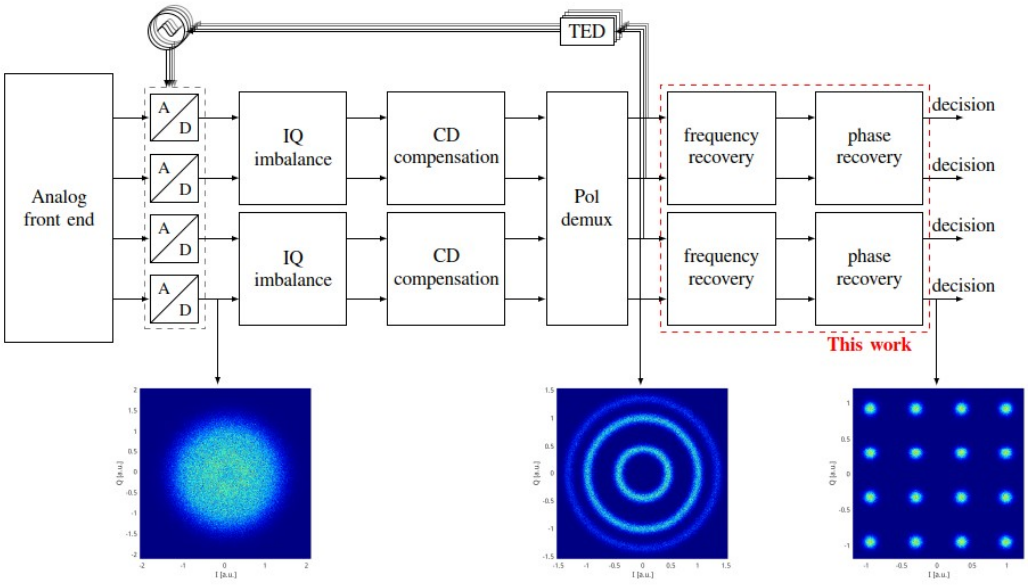



Figure 1.2: Full receiver DSP chain

Because of this position in the processing chain, its performance has a direct impact on the overall receiver quality. The implementation focuses not only on functional correctness, but also on efficient hardware mapping, pipelining and memory usage.

Where multiple implementation strategies are possible, the motivation behind the chosen design decisions will be discussed. Alternative approaches will also be considered, together with the reasons why they were not selected in the final design.

Finally, the performance of the implemented architecture will be evaluated. The proposed carrier recovery system contains several configurable parameters, such as the number of blind phase search stages, block size and phase-correction segments. Increasing these parameters can improve phase estimation accuracy and cycle slip tolerance, but also increases the required hardware resources, latency and power consumption.

The objective of this evaluation is therefore not only to determine the absolute performance of the design, but also to investigate the trade-off between hardware cost and receiver performance under different operating conditions. These conditions include different signal-to-noise ratios, residual frequency offsets and laser linewidths, as these are the dominant impairments affecting coherent carrier recovery.

## 1.4 Contributions

keep this for after main body has been written

This thesis presents the design and implementation of a low-complexity phase spread estimation architecture for coherent communication systems.

The main contributions of this work are:

- A fixed-point hardware implementation of the spread estimation algorithm suitable for FPGA deployment.
- The design of a pipelined architecture for phase estimation, including dot-product, reciprocal multiplication, and arccos lookup blocks.
- The development of filtering techniques to improve estimation stability, including moving average filtering and sign majority voting.
- A verification framework using Python reference models and VHDL testbenches to validate the fixed-point implementation.
- A performance evaluation of the proposed architecture in terms of numerical accuracy, latency, and resource usage.

## 1.5 Thesis outline

The remainder of this thesis is structured as follows:

Chapter 2 introduces the theoretical background required for this work. First, coherent optical communication systems are discussed, with a focus on how phase noise and residual frequency offset arise in these systems and how they affect the received signal. This is necessary to understand why carrier recovery is required and what challenges it introduces. Next, existing carrier recovery techniques are reviewed, together with their limitations and motivation for the proposed approach. The digital representation of 16QAM signals is then discussed, since the proposed architecture strongly depends on fixed-point symbol scaling, numerical precision and signal-to-noise ratio (SNR)-related considerations. Finally, a section is dedicated to the coordinate rotation digital computer (CORDIC) algorithm. Unlike the other algorithmic blocks in this thesis, which are developed specifically for this work, the CORDIC is based on an existing intellectual property (IP) core developed by [INSERT REF].

insert ref CORDIC IP

Chapter 3 presents the proposed phase recovery algorithm from a theoretical perspective. Building on the concepts introduced in the previous chapter, the complete architecture is divided into its main functional blocks, including the serialised binary phase search (SBPS), spread estimation (SE), phase unwrapper (PU) and phase compensator (PC) modules. For each subblock, the operating principle, mathematical model and expected contribution to the overall system are discussed. The theoretical performance of the proposed method is also analysed, based on the work of J. De Busscher in [INSERT REF].

insert ref PAPER JONAS

Chapter 4 describes how the theoretical carrier recovery system is translated into an FPGA implementation. This chapter discusses the practical challenges associated with such an implementation, including fixed-point numerical representations, timing closure, and hierarchical design. In addition, it explains how the individual subblocks are combined into a complete processing chain and how dataflow, valid signals and latency alignment are handled throughout the design.

Chapter 5 explains the simulation and verification framework used for the implemented design. The testbenches of the different subblocks are discussed together with their corresponding results. The purpose of this chapter is to show that the hardware implementation behaves in accordance with the theoretical model and to explain any deviations between both. The focus is therefore on functional correctness rather than on overall system performance under different operating conditions. Comparisons between the FPGA implementation and the Python or Simulink reference models are also included to demonstrate that the implemented fixed-point design accurately reproduces the expected behaviour.

Chapter 6 evaluates the overall performance of the implemented carrier recovery system. Separate sections discuss the performance of the SBPS and SE blocks and identify the conditions under which their performance begins to degrade. The influence of different system parameters, such as block size, number of search stages and datapath-bitlengths, is also investigated. In addition, this chapter discusses the hardware resource usage and latency of the final design. The relation between implementation complexity and system performance is analysed in order to determine which parameters offer the most favourable trade-off.

Chapter 7 discusses the limitations of the current implementation and identifies possible improvements for future work. This includes both algorithmic improvements and hardware-related optimizations. Potential extensions such as more advanced look-up table (LUT) implementations, adaptive parameter selection, improved spread estimation for low SNR conditions and support for higher-order modulation formats are considered. In addition, the discussion reflects on the assumptions made throughout the work, such as the assumption that the residual frequency offset changes only slowly between neighbouring blocks. The chapter also provides a broader overview of the main design trade-offs and explains how these trade-offs influenced the final architecture.

find good alternative solutions, these are purely speculative

Finally, Chapter 8 summarizes the most important aspects of the work. It revisits the original objectives of the thesis, discusses the extent to which these objectives were achieved and reflects on the main findings of the implementation and evaluation. The final chapter also outlines possible directions for future work, including integration into a complete coherent receiver chain and further optimization of the architecture for practical deployment.

# 2

# Background

This chapter summarises the background required to contextualise the proposed coherent receiver carrier-recovery architecture. It focuses on (i) short-reach IM/DD links versus coherent detection, (ii) phase-related impairments (carrier frequency offset and laser phase noise), and (iii) representative DSP-based carrier recovery methods used in coherent receivers

## 2.1 Coherent optical communication systems

Traditional short-reach optical interconnects in data centres have often favoured intensity-modulation/direct-detection (IM/DD) transceivers because they enable simple and power-efficient receiver front-ends. In IM/DD systems, information is conveyed by modulating the received optical intensity (or optical power). At the receiver, a photodiode converts incident optical power into an electrical photocurrent (square-law photodetection), so the receiver directly observes intensity but does not directly access the optical carrier phase [3, 4].

IM/DD links are attractive for short distances and moderate data rates; however, as symbol rates increase, fibre chromatic dispersion becomes increasingly detrimental because group delay varies with optical wavelength, causing waveform distortion and frequency-selective fading for intensity-modulated signals. In addition, the achievable spectral efficiency (information rate per unit bandwidth, in b/s/Hz) is fundamentally constrained when only intensity is detected, because direct detection effectively exploits fewer degrees of freedom than coherent detection [4].

Coherent optical communication systems overcome these limitations by detecting both the amplitude and phase of the optical field and enabling advanced modulation formats such as QPSK and square M-QAM. Coherent detection also supports digital compensation of linear channel impairments (e.g., chromatic dispersion and polarization effects) in the electrical domain [4, 5, 6]. Recent work on data-centre optics evaluates when coherent detection becomes favourable relative to IM/DD as per-lane rates scale, highlighting the trade-off between coherent DSP power/complexity and improved spectral efficiency and impairment tolerance [7].

A typical intradyne coherent receiver combines the received optical field with a local oscillator (LO) laser in a 90° optical hybrid. The hybrid generates orthogonal interference terms that are detected by balanced photodiode pairs, yielding electrical in-phase (I) and quadrature (Q) components. Balanced detection improves sensitivity to the desired beating term and suppresses common-mode intensity noise [5, 8].

After analog-to-digital conversion, the coherent receiver typically performs several DSP operations, including chromatic dispersion compensation, timing recovery, adaptive equalization, carrier frequency recovery and carrier phase recovery [5, 6]. In many DSP chains, dispersion compensation and polarization demultiplexing/equalization are performed prior to carrier phase estimation because commonly used blind equalizers can converge without an explicit carrier-phase reference; in practice, frequency-offset estimation may be placed either before or after equalization depending on acquisition range and implementation constraints [5, 6].

Historically, optical coherent receivers often relied on analogue phase-locked loops (PLLs) for carrier synchronisation. In modern DSP-based coherent receivers, carrier recovery is commonly implemented digitally using feed-forward and/or decision-directed algorithms, which avoids tight analogue loop dynamics and integrates naturally with polarization-diverse digital processing [5, 9, 10].

## 2.2 Phase noise and frequency offset

The main phase-related impairments in coherent optical communication systems are carrier frequency offset (CFO) and phase noise. Both arise because the transmitter and local oscillator lasers are independent devices and therefore do not share an identical optical frequency or phase reference [8].

Carrier frequency offset occurs because the instantaneous optical frequency of the transmitter laser differs from that of the LO laser. After coherent detection and digitisation, CFO appears as an effective baseband frequency offset and produces a deterministic phase ramp across samples [8]. Mathematically, this phase rotation can be expressed as

$$y[k] = s[k]e^{j(\theta[k]+2\pi\Delta f T_s k)} + \eta[k] \tag{2.1}$$

where $s[k]$ is the transmitted symbol, $\theta[k]$ is the phase noise, $\Delta f$ is the carrier frequency offset, $T_s$ is the symbol period and $\eta[k]$ is additive noise [8, 11].

The term $2\pi\Delta f T_s k$ corresponds to a linear phase ramp. A larger frequency offset or a larger symbol duration leads to faster constellation rotation. This is particularly relevant for block-based carrier recovery systems, because the total accumulated phase change within one block influences the probability of phase ambiguities and cycle slips [10].

Besides frequency offset, coherent optical systems are also affected by phase noise. Laser phase noise is commonly modelled as a discrete-time Wiener process (random walk) [11]:

$$\theta[k] = \theta[k-1] + \Delta\theta[k] \tag{2.2}$$

where $\Delta\theta[k]$ is a zero-mean Gaussian increment whose variance is proportional to the combined (beat) linewidth and the sampling interval [11, 10].

For the systems considered in this thesis, the residual CFO after coarse frequency recovery is especially important. While a separate frequency recovery stage can compensate most of the CFO, a small leftover frequency error remains due to the finite resolution of practical estimators [12, 13]. This residual offset causes a nearly linear phase drift across each DSP block.

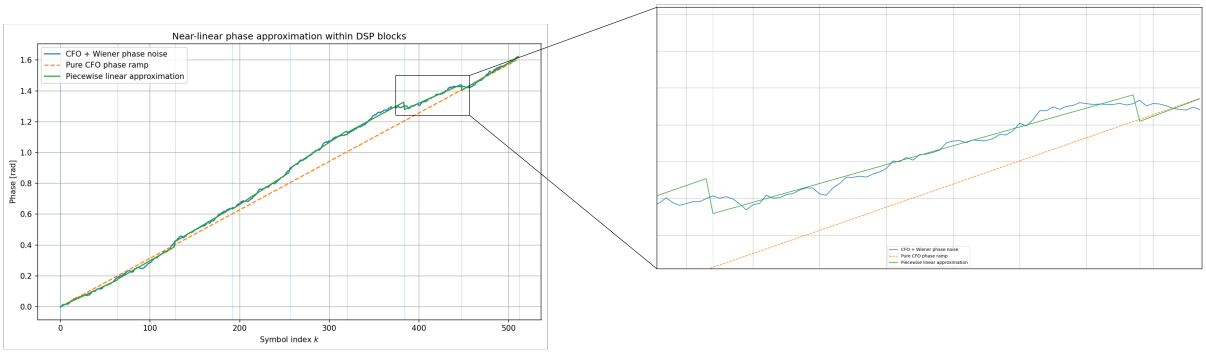



Figure 2.1: Near-lineair approximation of the total noise process ($\Delta f = 16$MHz, dnu = 0.1MHz, baud = 32GBaud)

clean up figure, legend

The phase recovery system must therefore not only compensate random phase noise caused by laser linewidth, but also account for remaining within-block phase slope. This motivates the channel model used throughout this thesis, in which the received symbols are impaired by:

- additive white Gaussian noise,
- a residual carrier frequency offset,
- Wiener phase noise caused by laser linewidth.

A coarse FFT-based frequency recovery algorithm is often used before phase recovery. FFT-based CFO estimation provides wide acquisition range at modest complexity [12, 13]. For M-PSK constellations, an M-th power nonlinearity can suppress the data modulation and expose a strong carrier-related spectral line that enables peak-based CFO estimation; related techniques are also adapted for square QAM via partitioning, pilots, or data-aided processing [14].

The estimated frequency offset can then be compensated digitally. However, the accuracy of this estimate depends on the FFT size. A finite FFT resolution leads to a remaining frequency estimation error. Since the FFT bin spacing equals $F_s/N_{\mathrm{FFT}}$, a simple bin-centred peak estimate implies an error on the order of half a frequency bin:

$$\Delta f_e \leq \frac{F_s}{2N_{\mathrm{FFT}}} \tag{2.3}$$

[15, 16].

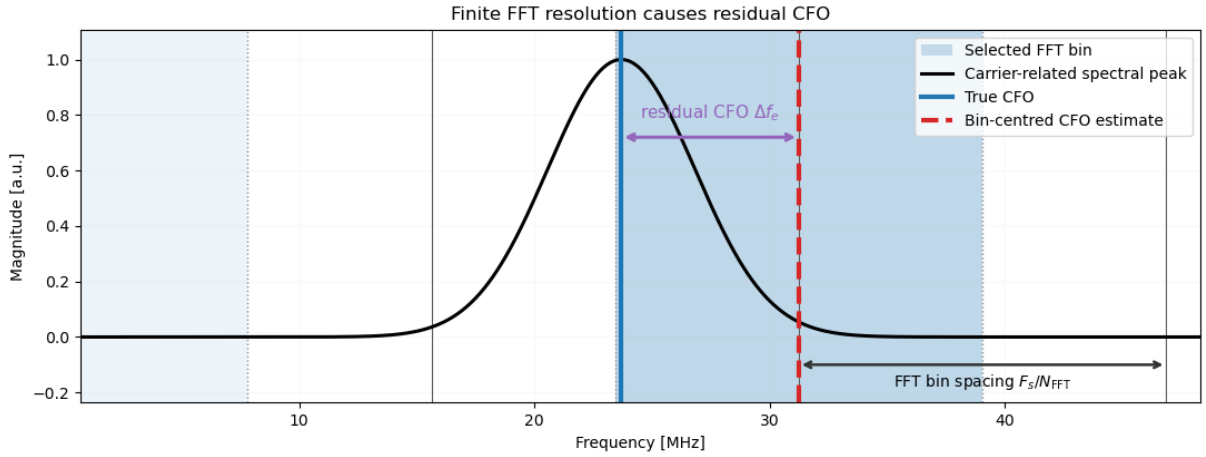



Figure 2.2: FFT-binsize leading to residual phase offset ($F_s = 32$GHz, $N_{FFT} = 2048$)

This remaining frequency error is then passed on to the phase recovery stage. If the residual offset is too large, the phase recovery block may no longer be able to track the constellation rotation correctly, which increases the probability of cycle slips [10, 6].

## 2.3 Carrier recovery in DSP chains

Carrier recovery is the DSP stage responsible for compensating the phase rotations introduced by residual CFO and phase noise. Since these impairments have different characteristics, coherent receivers commonly separate carrier recovery into (i) frequency offset estimation/compensation and (ii) carrier phase estimation [5, 6].

Frequency recovery estimates a single CFO parameter over an observation interval and therefore benefits from aggregating many samples to improve estimator variance; it is not typically meaningful to produce an independent CFO estimate per symbol. By contrast, carrier phase recovery must track a time-varying phase process (Wiener phase noise plus residual CFO), motivating symbol-rate or block-rate phase estimation[12, 10].

A major challenge is that modern coherent DSP systems process many symbols in parallel in order to reduce clock speed requirements. As a result, carrier phase is often estimated only once per processing block instead of once per symbol [6]. Although this significantly reduces computational complexity, it also increases sensitivity to phase drift within the block. If the total phase change over a block becomes too large, the phase recovery algorithm may unwrap to an incorrect $\pi/2$-shifted hypothesis (cycle slip) for square QAM constellations [10].

If a block contains $B$ symbols and the residual frequency offset is $\Delta f_e$, the total phase drift across the block can be approximated by

$$\Delta\phi_{\mathrm{block}} \approx 2\pi\Delta f_e B T_s \tag{2.4}$$

To reduce the probability of cycle slips, a practical design rule is to keep the intra-block phase drift well below the midpoint between neighbouring $\pi/2$-ambiguous hypotheses, often expressed as $|\Delta\phi_{\mathrm{block}}| \lesssim \pi/4$ [10].

## 2.4 Blind Phase Search

Blind Phase Search (BPS) is a widely used feed-forward carrier phase estimation method for coherent square M-QAM. Its advantages include pilot-free operation, modulation-format compatibility, and strong tolerance to laser phase noise when sufficient test phases and averaging are used [10, 17].

The principle of BPS is straightforward. A block of received symbols is rotated over a set of candidate test angles. For each candidate angle, the rotated symbols are sliced to the nearest ideal constellation points and an error metric is computed. The candidate angle that minimises the total error is selected as the estimated carrier phase [10].

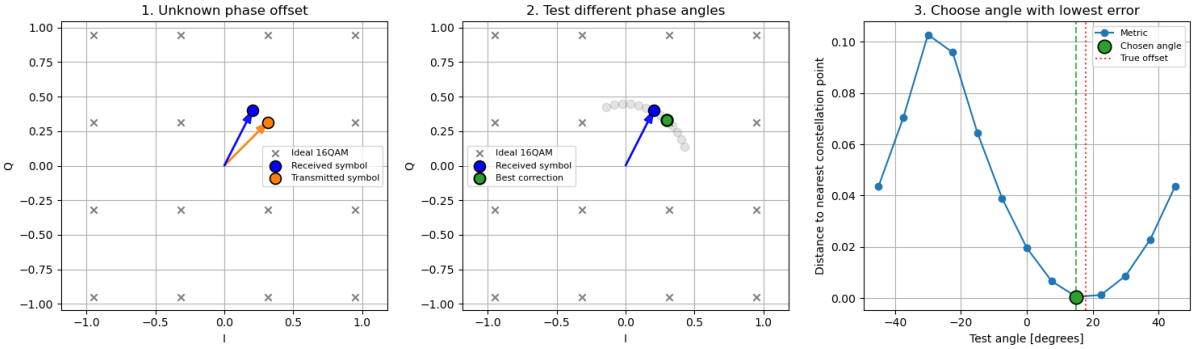



Figure 2.3: Working BPS principle

For a received block $z[k]$, the BPS metric for a test phase $\phi$ can be written as

$$e(\phi) = \sum_{k=0}^{B-1} \left| D\left(z[k]e^{-j\phi}\right) - z[k]e^{-j\phi} \right|^2 \tag{2.5}$$

where $D(\cdot)$ denotes the decision operation that maps a rotated symbol to its nearest constellation point. The phase estimate is then given by the test angle that minimises $e(\phi)$.

For square QAM constellations, the objective function in BPS is periodic with period $\pi/2$ due to constellation rotational symmetry. Therefore, it suffices to search any interval of length $\pi/2$; a common centred choice is $[-\pi/4, \pi/4]$ [10]. However, a large number of test phases is usually required to obtain sufficient estimation accuracy, making BPS computationally expensive in high-throughput FPGA implementations [10, 17].

Another limitation of blockwise BPS is that it estimates one average phase per block. When residual CFO induces significant intra-block phase drift, a single blockwise estimate can be insufficient and may increase cycle-slip probability [10]. This thesis therefore extends the traditional blockwise BPS approach by also estimating the phase spread across the block, as explained in Chapter 3.

## 2.5 CORDIC

The Coordinate Rotation Digital Computer (CORDIC) algorithm implements vector rotations and elementary functions using iterative shift-add operations. Its attractiveness in FPGA/ASIC implementations stems from replacing general multipliers with pipelinable add/subtract and bit-shift datapaths [18, 19].

In the rotation mode of CORDIC, a vector is iteratively rotated by a sequence of predefined micro-rotations. The iterative equations can be written as [18, 19]:

$$x_{i+1} = x_i - d_i y_i 2^{-i} \tag{2.6}$$

$$y_{i+1} = y_i + d_i x_i 2^{-i} \tag{2.7}$$

$$z_{i+1} = z_i - d_i \arctan(2^{-i}) \tag{2.8}$$

where $d_i \in \{-1, +1\}$ determines the direction of rotation at iteration $i$. After a sufficient number of iterations, the vector is rotated by the desired angle with good precision [19].

In this thesis, CORDIC is used to rotate received symbols by the estimated carrier phase. The implementation leverages an FPGA IP core; design effort therefore focuses on selecting fixed-point formats, pipeline depth/latency, and interface timing to meet throughput and numerical-accuracy requirements [20].

*This page is intentionally left blank.*

# 3

# Proposed phase recovery architecture

## 3.1 System overview

As discussed in Chapter 2, coherent optical receivers are affected by both residual carrier frequency offset (carrier frequency offset (CFO)) and laser phase noise. A coarse frequency recovery stage removes the majority of the frequency offset, but practical estimators always leave a remaining residual error because of their finite frequency resolution. This residual CFO introduces a deterministic phase rotation across consecutive symbols, while the finite linewidth of the transmitter and local oscillator lasers adds a stochastic phase perturbation on top of this process.

In conventional feed-forward carrier recovery systems, a single carrier phase estimate is often calculated over a block of symbols. This significantly reduces computational complexity compared to per-symbol phase estimation, which is especially important in high-throughput coherent receivers where many symbols are processed in parallel. However, this also introduces an important limitation: the carrier phase is generally not constant over the duration of the processing block.

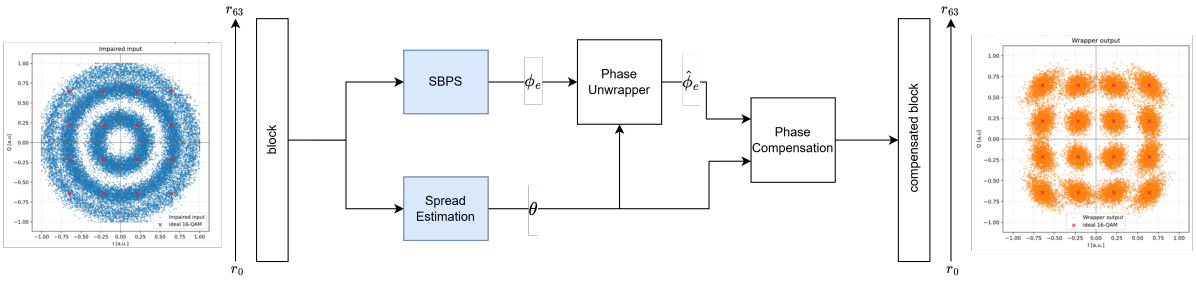



Figure 3.1: Top architecture

make figure more clear, seems to minimalistic?

In the presence of residual CFO, the phase evolves approximately linearly with symbol index, while laser phase noise introduces additional random deviations. As a result, a single phase estimate can become insufficient for accurately compensating all symbols inside the block. Symbols located further away from the phase estimation point accumulate additional residual phase error.

The proposed architecture addresses this problem through a two-part carrier recovery structure consisting of:

1. a Serialized Blind Phase Search (SBPS) block,
2. and a Spread Estimation (SE) block.

The SBPS estimates the carrier phase near the middle of the processing block and acts as the central phase reference of the architecture. The SE block then estimates how the phase evolves around this midpoint estimate. Rather than estimating a separate phase for every symbol, the architecture reconstructs the intra-block phase evolution from:

- one centre-phase estimate,
- and one estimate of the total phase spread across the block.

This approach is based on the observation that, under practical coherent datacenter operating conditions, the residual CFO after coarse frequency recovery dominates the short-term phase evolution within one processing block. The accumulated linewidth-induced phase variation over a single block is typically much smaller than the deterministic phase drift caused by the residual frequency offset:

$$\sum_{i=0}^{B-1} \phi_i \ll 2\pi \Delta f T_s B \tag{3.1}$$

with $B$ the block size, $\phi_i$ the linewidth-induced phase perturbation of symbol $i$, $\Delta f$ the residual carrier frequency offset and $T_s$ the symbol period.

Under this assumption, the phase evolution across the block may be approximated as locally linear. The carrier phase inside the block can therefore be reconstructed from only:

1. the carrier phase near the middle of the block,
2. and the total phase spread across the block.

Figure 3.4 illustrates the effect that motivates this additional spread estimate. Although the middle of the block can be compensated correctly by the SBPS estimate, symbols towards the block edges still exhibit residual constellation rotation when the intra-block phase evolution is ignored.

The estimated spread is subsequently used in two ways:

1. to improve phase unwrapping between neighbouring blocks,
2. and to reconstruct multiple phase corrections within the block during phase compensation.

As a result, the proposed architecture increases the effective frequency tracking range of the carrier recovery system while maintaining substantially lower complexity than conventional high-resolution BPS approaches.

## 3.2 Serialized blind phase search

In a traditional BPS implementation, many candidate phases are evaluated simultaneously over the full search interval. For each candidate phase, all symbols inside the processing window are rotated, sliced to the nearest constellation point and evaluated through a decision-directed metric. The candidate yielding the lowest metric is selected as the estimated carrier phase.

For square quadrature amplitude modulation (QAM) constellations, the search interval may be limited to:

$$\left[-\frac{\pi}{4}, \frac{\pi}{4}\right)$$

because the BPS metric is inherently $\pi/2$-periodic due to the rotational symmetry of square constellations .

is source already cited?

The key idea of SBPS is that the optimal phase estimate is approximated through successive refinement stages. Instead of evaluating many test phases simultaneously, only two candidate rotations are evaluated at each stage:

- one positive correction,
- and one negative correction,

relative to the previously selected estimate.

The candidate producing the lower metric is retained, after which a more refined correction is evaluated during the next stage.

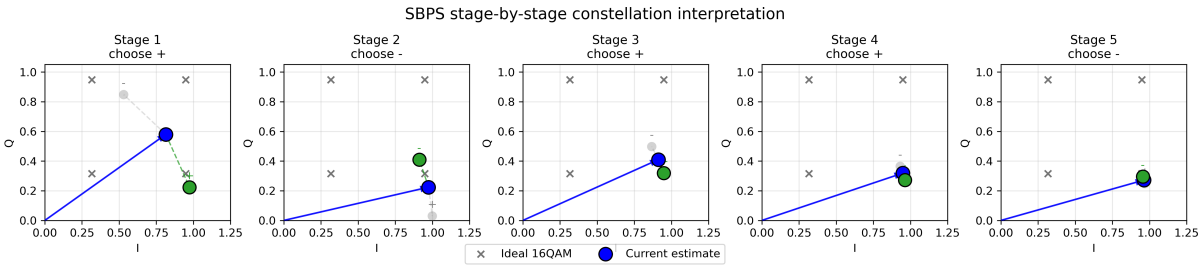



Figure 3.2: Conceptual representation of staged SBPS refinement

The proposed implementation adopts a binary refinement structure. The first stage evaluates the hypotheses:

$$\phi_1 \in \left\{-\frac{\pi}{8}, +\frac{\pi}{8}\right\}$$

while stage $m$ refines the estimate through:

$$\hat{\phi}_m = \hat{\phi}_{m-1} \pm \frac{\pi}{2^{m+2}}$$

Each stage therefore halves the remaining search uncertainty.

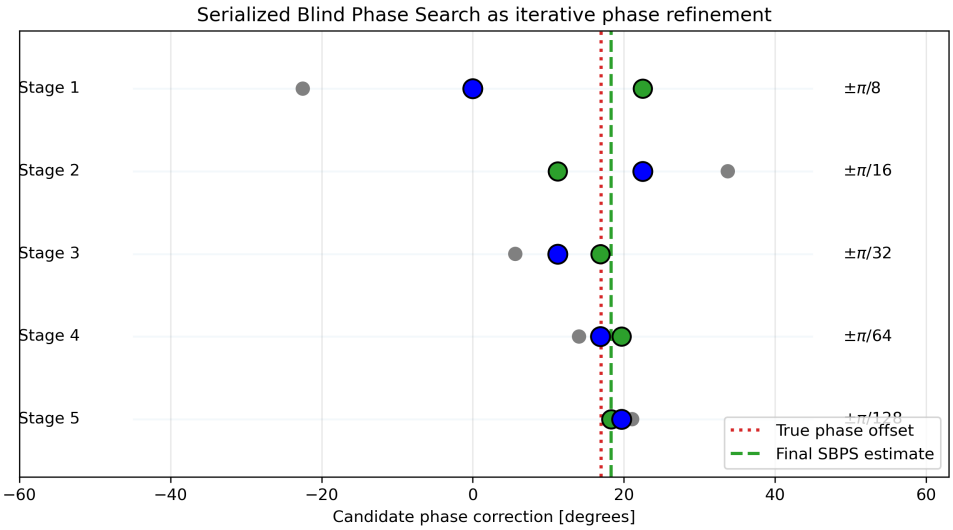



Figure 3.3: Phase representation of staged SBPS refinement

This procedure is mathematically equivalent to a binary search over the phase interval and yields an exponentially decreasing phase quantization error as the number of stages increases. The referenced work shows that the worst-case estimation error decreases exponentially with each added stage, while the total number of required candidate rotations only increases linearly with the number of stages [21].

For each candidate phase $\phi$, a decision-directed metric is computed:

$$J(\phi) = \sum_{k \in \mathcal{W}} \left| D\left(r_k e^{-j\phi}\right) - r_k e^{-j\phi} \right|^2$$

with:

- $r_k$ the received symbol,
- $D(\cdot)$ the slicer operation,
- and $\mathcal{W}$ the metric accumulation window.

The candidate phase producing the smallest metric is assumed to correspond most closely to the true carrier phase. In this work, the resulting estimate is interpreted as the carrier phase near the middle of the processing block $\hat{\phi}_e$.

This estimate is subsequently forwarded to the phase unwrapping and phase compensation stages. Because only one phase estimate is produced per block, and because the BPS metric is $\pi/2$-ambiguous for square QAM constellations, additional processing is required to track phase continuity between successive blocks. This task is handled by the phase unwrapping stage, while the SE block reconstructs the phase evolution inside each block.

### 3.2.1 Block size consideration

The block size directly influences the effectiveness of the SBPS estimate. The coarse frequency recovery stage has a finite fast fourier-transform (FFT)-bin spacing, which determines the maximum residual CFO after frequency recovery. Using the half-bin error bound from Equation 2.3, the residual frequency error satisfies:

$$\Delta f_e \leq \frac{F_s}{2N_{\mathrm{FFT}}} \tag{3.2}$$

This residual frequency error causes an accumulated phase drift across one SBPS processing block of $B$ symbols:

$$\Delta\phi_{\mathrm{block}} \approx 2\pi\Delta f_e B T_s \tag{3.3}$$

Since $T_s = 1/F_s$, substituting the worst-case residual CFO gives:

$$\Delta\phi_{\mathrm{block,max}} = 2\pi\left(\frac{F_s}{2N_{\mathrm{FFT}}}\right)B\left(\frac{1}{F_s}\right) = \pi\frac{B}{N_{\mathrm{FFT}}} \tag{3.4}$$

The worst-case accumulated phase drift over one block therefore depends on the ratio between the processing block size and the FFT size, and not directly on the sampling frequency. For an unwrapping threshold of $\pi/4$, the phase drift over one block should satisfy:

$$\Delta\phi_{\mathrm{block,max}} \leq \frac{\pi}{4} \tag{3.5}$$

which yields the design condition:

$$N_{\mathrm{FFT}} > 4B \tag{3.6}$$

For example, a block size of $B = 64$ requires at least $N_{\mathrm{FFT}} > 256$ to avoid additional cycle slips from the worst-case coarse frequency recovery error. This is only a theoretical minimum. In practice, a margin is required because linewidth-induced phase noise, additive noise and estimation errors also affect the phase trajectory.

The block size also creates a trade-off in estimation quality. A larger metric accumulation window improves robustness against additive noise because random perturbations are averaged over more samples [22]. At the same time, a larger block spans a longer time interval, so the carrier phase variation caused by residual CFO and laser phase noise becomes larger within the block. This trade-off is one of the main motivations for combining a middle-block SBPS estimate with a separate SE block.

## 3.3 Spread estimation algorithm

As discussed in Sections 2.2 and 2.3, the residual impairments after coarse frequency recovery consist of both stochastic laser phase noise and a remaining carrier frequency offset. While the linewidth-induced phase noise behaves as a random Wiener process, the remaining frequency offset introduces an approximately linear phase ramp across a processing block, as illustrated in Figure 2.1.

The proposed SBPS architecture estimates only a single carrier phase per block, namely the phase located near the middle of the block. This estimate is sufficiently accurate to serve as a robust phase reference for the block as a whole, but it does not describe how the phase evolves within the block itself. As a result, directly compensating all symbols using only the middle-block phase estimate still leaves a residual phase error on symbols located further away from the block centre.

This effect becomes increasingly important when the residual CFO after frequency recovery becomes larger, when the processing block size increases or when the laser linewidth introduces additional phase drift.

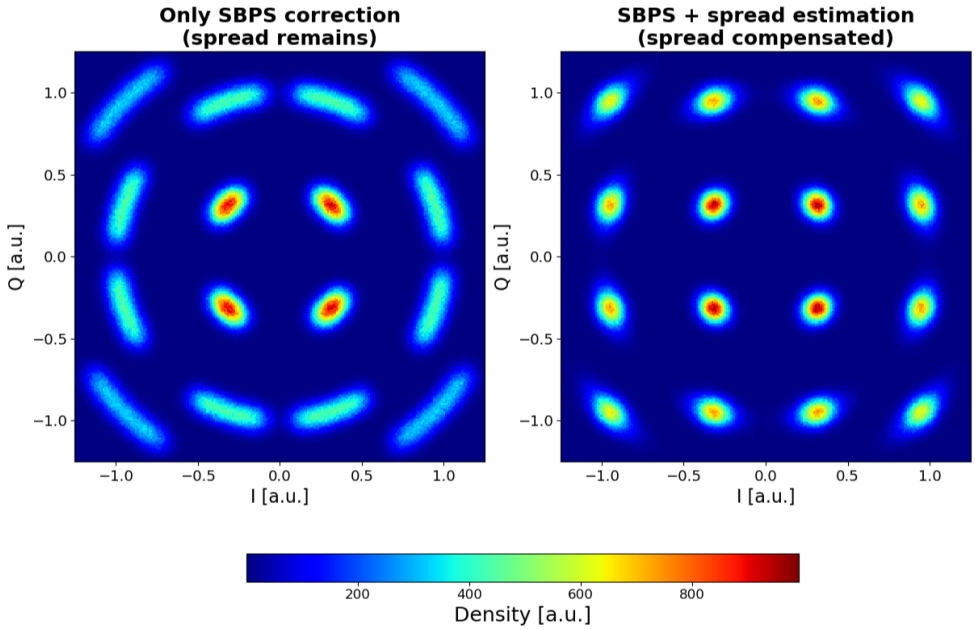



Figure 3.4: Residual phase correction with SBPS only and with combined SBPS+SE correction

The SE block is introduced to estimate this residual phase evolution across the block. Rather than estimating a separate carrier phase for every symbol, which would significantly increase computational complexity, the proposed method estimates the total phase spread over the block and reconstructs the intermediate phase trajectory from this information.

The proposed spread estimation algorithm operates on symbols that are raised to the fourth power. For square QAM constellations, the fourth-power operation suppresses the modulation-dependent phase terms while preserving the carrier-related phase rotation.

For a received symbol:

$$r_k = a_k e^{j\phi_k} \tag{3.7}$$

the fourth-power operation gives:

$$r_k^4 = a_k^4 e^{j4\phi_k} \tag{3.8}$$

The modulation phase ambiguity is removed because square QAM constellations exhibit rotational symmetry over multiples of $\pi/2$. After fourth-power transformation, the dominant remaining phase term therefore corresponds to four times the underlying carrier phase evolution.

Under the locally linear phase assumption introduced in Equation 3.1, the total phase spread across the block may be estimated from representative symbols located near:

- the beginning of the block,
- the middle of the block,
- and the end of the block.

The first and last symbols are used to estimate the magnitude of the total phase spread, while the middle symbol assists in determining the direction of the phase evolution.

Let $R_0$ and $R_{B-1}$ denote the selected fourth-power symbols near the beginning and end of the block. The cosine of the total phase spread may then be estimated through the normalized dot product:

$$\frac{\langle R_0, R_{B-1} \rangle}{\|R_0\| \|R_{B-1}\|} = \cos(4|\theta|) \tag{3.9}$$

with $\theta$ the total phase spread across the block.

The spread magnitude can therefore be recovered through:

$$|\theta| = \frac{1}{4}\arccos\left(\frac{\langle R_0, R_{B-1}\rangle}{\|R_0\|\|R_{B-1}\|}\right) \tag{3.10}$$

The use of $\arccos(\cdot)$ instead of $\arctan(\cdot)$ is important because the total phase spread may exceed $\pi$. A direct arctangent-based angle estimation could therefore lead to sign ambiguities and incorrect unwrap decisions. The cosine-based formulation instead estimates only the absolute spread magnitude, after which the spread direction is estimated separately.

To determine the direction of the phase evolution, a second estimate is performed using symbols from the beginning and middle of the block. The sign of the phase evolution can be obtained through the sign of the complex cross product:

$$\text{sign}(\gamma) = -\text{sign}\left(\Re\{R_0\}\Im\{R_b\} - \Im\{R_0\}\Re\{R_b\}\right) \tag{3.11}$$

with $R_b$ the selected middle-region symbol.

Combining both estimates yields the final spread estimate:

$$\hat{\theta} = \text{sign}(\gamma) \cdot |\theta| \tag{3.12}$$

The estimated spread is subsequently used to extrapolate leading and lagging phase estimates for the phase unwrapper and to reconstruct multiple phase corrections inside the block during phase compensation.

The method nevertheless introduces several limitations specific to square 16QAM constellations. Unlike quatrature phase shift keying (QPSK), where the fourth-power operation maps all transformed symbols onto a single radius, the fourth-power transformation of 16QAM results in multiple amplitude rings. Some transformed constellation points therefore become unsuitable for accurate spread estimation because their relative phase relationships introduce ambiguities.

As a consequence, not every received symbol may be used directly for spread estimation. The architecture therefore includes an input-selection stage that identifies suitable candidate symbols before the actual spread calculation is performed.

### 3.3.1 Input symbol consideration

For QPSK, the fourth-power operation maps all constellation symbols onto a single circle. For square 16QAM, however, the transformed symbols occupy multiple radii after the fourth-power operation. This behaviour is illustrated in Figure 3.5.

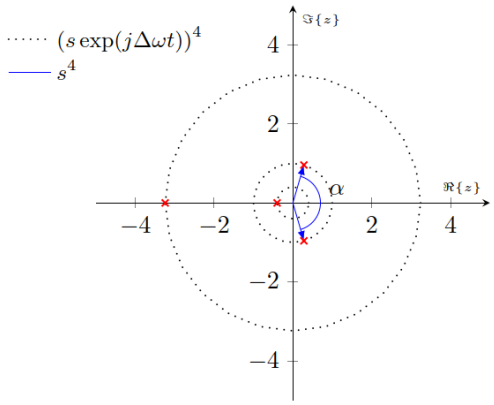



Figure 3.5: 16QAM symbols after fourth-power transformation

find more representative mapping

Symbols originating from the middle ring of the 16QAM constellation can introduce large phase ambiguities after fourth-power transformation. Using these symbols directly in the normalized dot-product calculation of Equation 3.9 can therefore lead to incorrect spread estimates. To mitigate this problem, the spread estimator only considers symbols originating from the inner and outer constellation

rings. These symbols retain an unambiguous phase transformation after the fourth-power calculation and therefore yield more reliable spread estimates.

Rather than searching the complete processing block for suitable symbols, only a limited subset of symbols near the beginning, middle and end of the block is evaluated. This reduces hardware complexity while maintaining a sufficiently high probability of finding valid candidate symbols.

Let $N$ denote the number of candidate symbols evaluated per region. Assuming uniformly distributed 16QAM symbols, the probability of finding at least one valid candidate pair may be approximated by:

$$P(\text{hit}) = 1 - \left[\left(\frac{1}{2^N}\right)^2 + 2\left(\frac{1}{2^N}\left(1 - \frac{1}{2^N}\right)\right)\right] \tag{3.13}$$

A moderate value of $N$ therefore already yields a high probability of obtaining usable symbols while avoiding the hardware cost associated with a full-block search. For example, $N = 3$ gives $P(\text{hit}) = 49/64 \approx 76.6\%$ under the uniform-symbol assumption. If no suitable candidate combination is found, the previously estimated spread value is retained. This behaviour is justified by the relatively slow variation of practical laser frequency offsets compared to the short duration of a processing block in baudrate-sampled coherent systems.

The selected candidate symbols are subsequently forwarded to the hardware blocks responsible for fourth-power calculation, spread estimation and sign estimation.

## 3.4 Phase unwrapping

## 3.5 Phase compensation

## 3.6 Expected performance

*This page is intentionally left blank.*

# 4

# Hardware implementation

Here you show: VHDL design bit widths latencies signals testbenches block diagrams

## 4.1 System overview

## 4.2 Data representation and scaling

explain why samples are in I/Q fxp, and testangles are cos/sin fxp (more difficult to calculate angles with, easier to apply angles to samples)

## 4.3 Serialized blind phase search block

The hardware implementation follows a staged refinement architecture, shown in Fig. 4.1. The same received block

$$\bar{r} = (r_0, r_1, \ldots, r_{\mathcal{W}-1})$$

is presented to every stage, while only the accumulated phase estimate is updated from stage to stage. At stage $m$, two candidate phase updates are evaluated around the previously selected estimate $\phi_{m-1}$, namely $\phi_{m-1} - \phi_t$ and $\phi_{m-1} + \phi_t$. The candidate that produces the smaller block error is selected as the stage output $\phi_m$, which is then forwarded to the next stage. By cascading several stages with progressively smaller test angles, the architecture converges towards the phase estimate associated with the centre of the block.

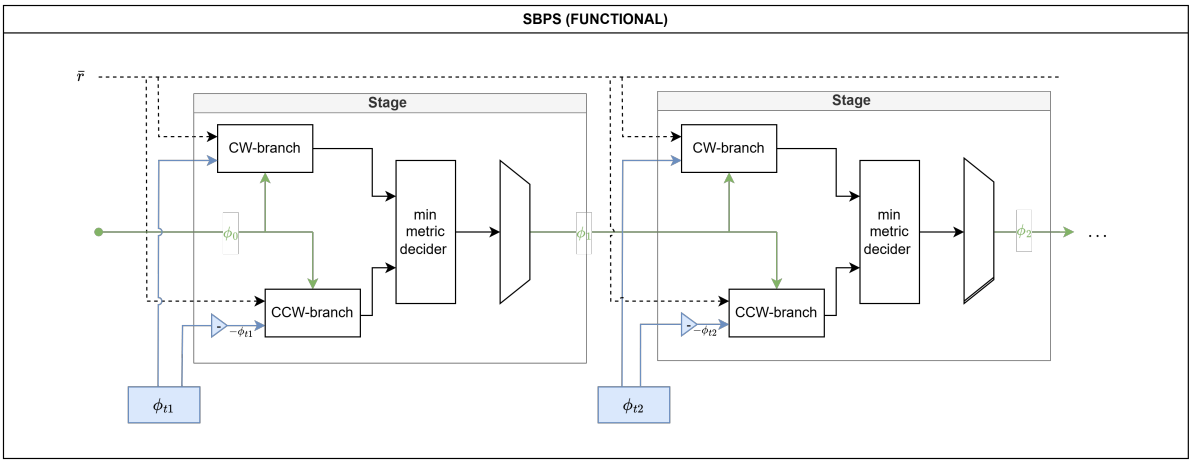



Figure 4.1: Functional diagram of the full SBPS architecture

### 4.3.1 Functional operation

Figure 4.2 shows the internal organisation of a single stage. The upper branch corresponds to the clockwise (CW) update and evaluates $\phi_{m-1} - \phi_t$, whereas the lower branch corresponds to the counter-clockwise (CCW) update and evaluates $\phi_{m-1} + \phi_t$. Each branch contains a *rotate metric slice* unit that processes the complete input block using its own candidate phase. The resulting block metrics are compared by the *min metric decider*, and the associated candidate phase is selected by the final multiplexer (MUX). In this way, each stage performs a binary phase refinement around the current estimate.

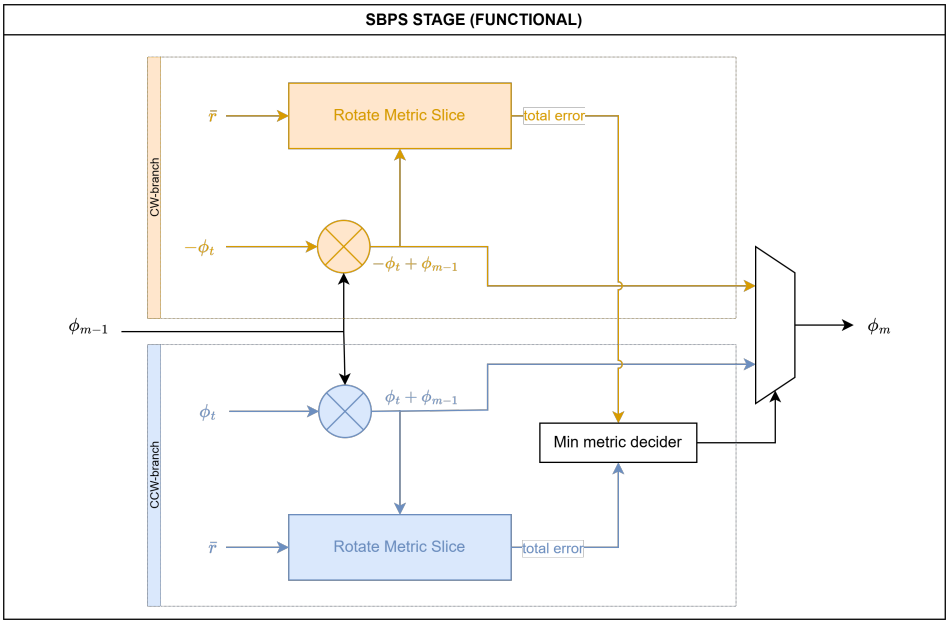



Figure 4.2: Functional diagram of the SBPS stage

The *rotate metric slice* unit is the core computational block of the design and is detailed in Fig. 4.3. For every symbol in the block, the candidate phase is used to rotate the received sample, the rotated sample is sliced to the nearest constellation point, and the squared Euclidean distance between the rotated and sliced values is computed. These per-symbol errors are then accumulated over the full block window to obtain a single decision metric for the branch. All symbols in the block are therefore processed in parallel inside each branch, while the serial refinement occurs across the SBPS stages.

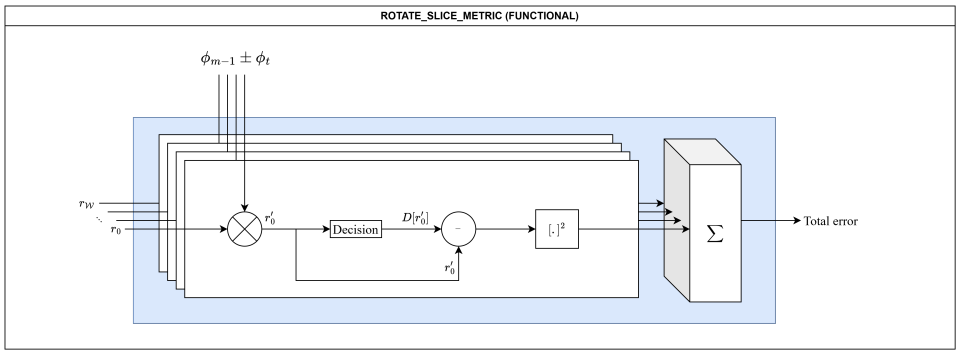



Figure 4.3: Functional diagram of the *rotate metric slice* unit

### 4.3.2 Data flow

The data-oriented view of the implementation is shown in Figs. 4.4, 4.5, and 4.6.

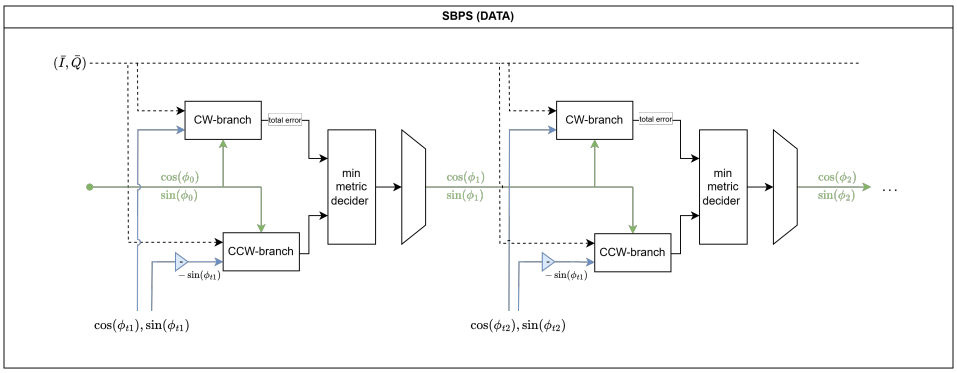



Figure 4.4: Datapath diagram of the full SBPS architecture

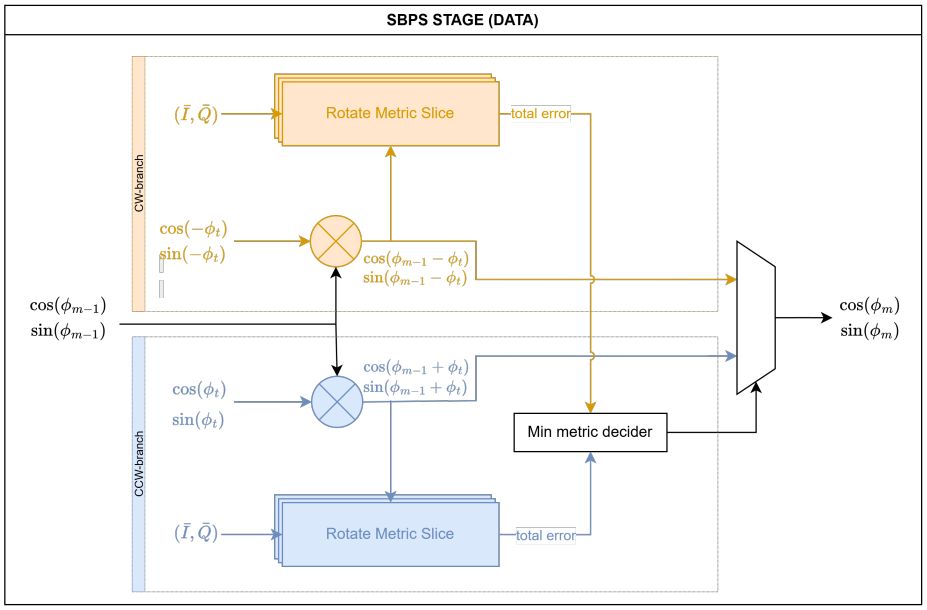



Figure 4.5: Datapath diagram of the SBPS stage

In the implemented datapath, phase is not represented as an explicit angle word. Instead, each phase value is carried by its trigonometric components $(\cos(\phi), \sin(\phi))$ in fixed-point format. This allows the candidate phases to be generated directly by the *angle add* block from the previous estimate $(\cos(\phi_{m-1}), \sin(\phi_{m-1}))$ and the stage test angle $(\cos(\phi_t), \sin(\phi_t))$:

$$\cos(\phi_{m-1} \pm \phi_t) = \cos(\phi_{m-1})\cos(\phi_t) \mp \sin(\phi_{m-1})\sin(\phi_t), \tag{4.1}$$
$$\sin(\phi_{m-1} \pm \phi_t) = \sin(\phi_{m-1})\cos(\phi_t) \pm \cos(\phi_{m-1})\sin(\phi_t). \tag{4.2}$$

Once a candidate phase has been formed, the received samples are rotated in the $I/Q$ plane. For a candidate angle

$$\theta \in \{\phi_{m-1} - \phi_t,\ \phi_{m-1} + \phi_t\},$$

the rotator computes

$$\begin{aligned} I'_k &= I_k \cos(\theta) + Q_k \sin(\theta), \\ Q'_k &= -I_k \sin(\theta) + Q_k \cos(\theta), \end{aligned} \tag{4.3}$$

for every symbol $r_k = I_k + jQ_k$ in the block. The rotated symbol is then denoted by $r'_k = I'_k + jQ'_k$.

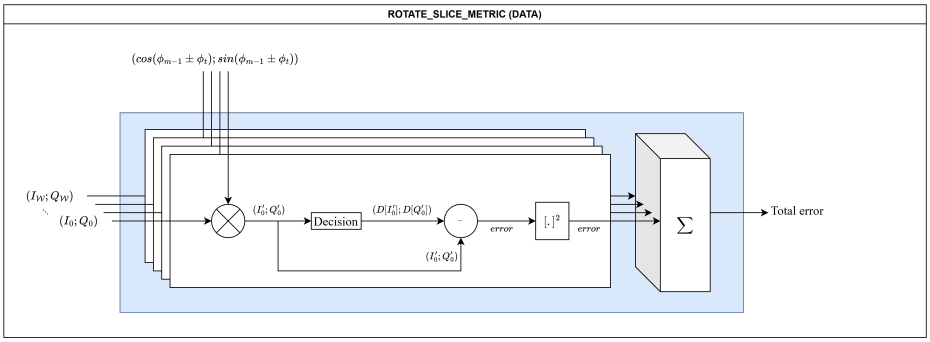



Figure 4.6: Datapath diagram of the *rotate metric slice* unit

For standard 16QAM operation, the slicer maps each rotated sample to the nearest reference point $D[r'_k]$. The axis decisions are derived from the conventional inner–outer thresholds, with $\pm 2s$ defining the transition between the inner and outer levels of the constellation. The resulting single-symbol error is

$$e_k = |r'_k - D[r'_k]|^2 = (I'_k - D_I[r'_k])^2 + (Q'_k - D_Q[r'_k])^2 , \tag{4.4}$$

and the branch metric is obtained by accumulation over the complete block:

$$E = \sum_{k=0}^{\mathcal{W}-1} e_k . \tag{4.5}$$

This accumulated metric is the quantity used by the *min metric decider* to select the surviving branch.

### 4.3.3 Timing flow

The timing behaviour of the full chain, of one SBPS stage, and of the *rotate metric slice* unit is summarised in Figs. 4.7, 4.8, and 4.9, respectively. The control path uses single-cycle valid pulses rather than a ready/valid handshake, so temporal alignment is achieved entirely through fixed pipeline delays.

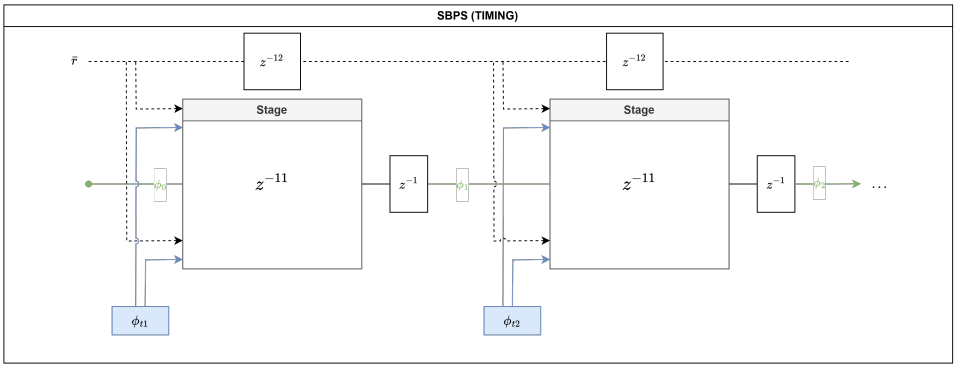



Figure 4.7: Timing diagram of the full SBPS algorithm

At the stage input, the two candidate phases are first generated while the input sample block is delayed in parallel so that both reach the branch datapaths in synchrony. The *angle add* operation introduces three elapsed clock cycles, and the sample block is delayed by the same amount before it enters the *rotate metric slice* unit.

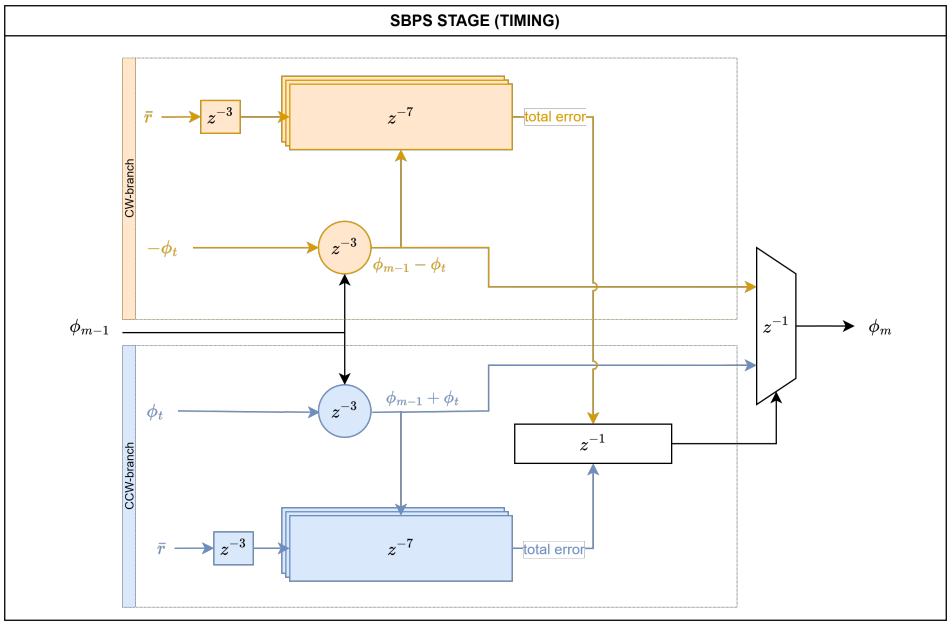



Figure 4.8: Timing diagram of the SBPS stage

Inside the *rotate metric slice* unit, the rotator contributes three cycles, the slicer and sample-alignment logic contribute one cycle, the single-symbol metric calculation contributes one cycle, and the summation plus output registration contribute two further cycles. The branch metric is therefore available seven cycles after entry into the *rotate metric slice* unit, or ten elapsed cycles after the stage launch when the initial angle-generation and input-alignment delay are included.

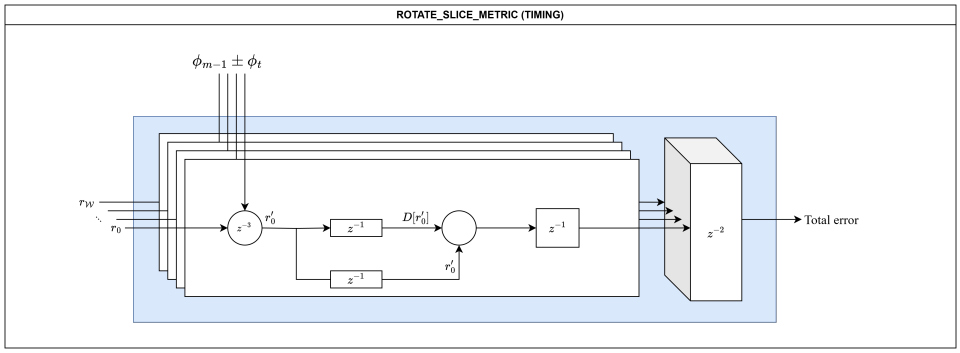



Figure 4.9: Timing diagram of the *rotate metric slice* unit

After both branch metrics have become available, the *min metric decider* requires one clock cycle to determine the winning branch, and the final stage output is registered one cycle later. Consequently, the phase-selection datapath of a stage spans eleven clock cycles, while the registered stage-valid output is asserted after twelve clock cycles. When latency is reported at the boundary of the complete SBPS chain rather than at a single stage boundary, one additional launch cycle must be included because the top-level chain registers the hand-off into the first stage.

### 4.3.4 16QAM-specific adaptations

For 16QAM, the slicer supports both conventional axis-based decisions and an optional radius-directed mode in the early refinement stages. In the conventional case, the decision on each axis is determined by the inner–outer threshold at $\pm 2s$. In the radius-directed variant, an additional threshold $l$ is introduced to identify samples that should be mapped directly to the outer ring of the constellation. This is beneficial in the first stages of the search, where the residual phase uncertainty is still relatively large and a purely axis-based decision can be less reliable. Once the phase estimate has been sufficiently refined, the later

stages revert to the standard 16QAM slicer, so that the final metric evaluation follows the nominal decision regions.

not well enough explained

## 4.4 Spread estimation block

The hardware implementation is split up in the different operational sections of the algorithm, namely the input selector (1), fourth-power calculators (2), the spread estimator (3) & the sign estimator (4) and finally the filters (5). The layout of these modules and their relative position is illustrated on Fig 4.10. Each section will individually discuss the modules in terms of its functional operation, dataflow and timing flow in order to get a clearer picture of the RTL implementation.

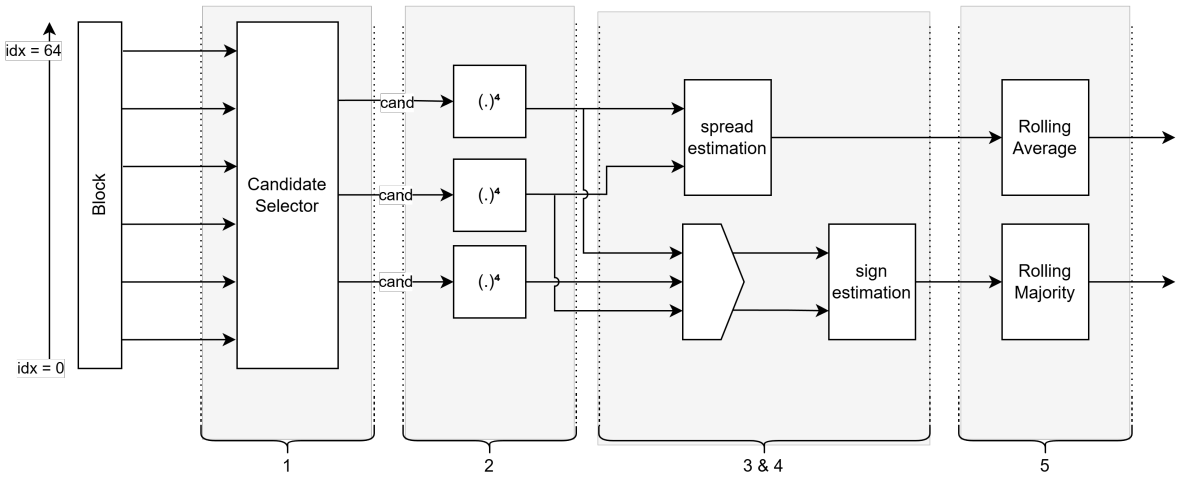



Figure 4.10: Top lay-out of SE block

### 4.4.1 Input selector

#### Functional operation

As introduced in Section 3.3.1, the spread and direction estimation blocks require a set of representative symbols from the input block. However, not all symbols are suitable for this purpose. Therefore, an input selection stage is required to extract valid candidate symbols from different regions of the block.

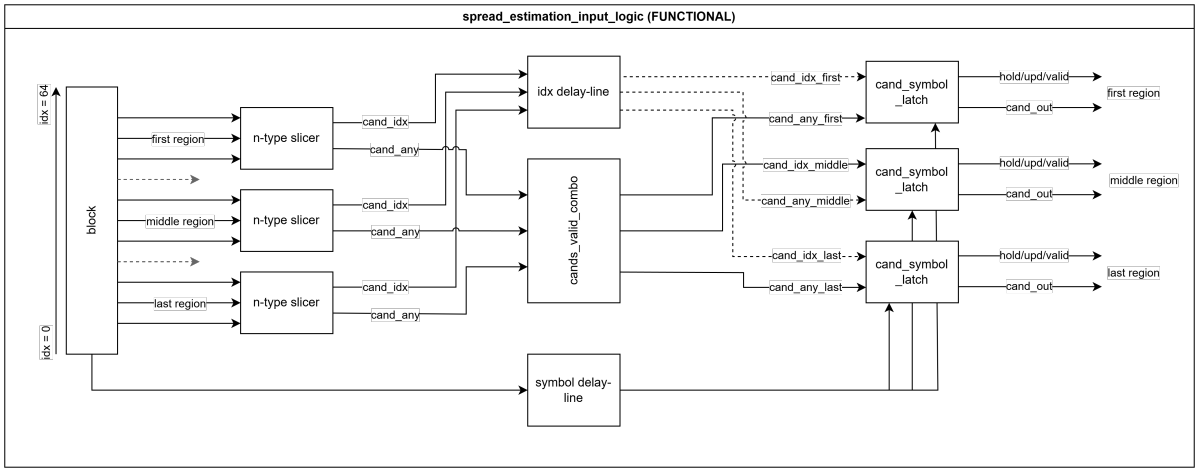



Figure 4.11: Functional diagram of the SE input selector

are signal big enough to be readable?

The input selector operates on the full block of received symbols $\bar{r}$, and outputs three candidate symbols $x$, $y$, and $z$, corresponding to the first, middle, and last regions of the block, respectively. These candidates are subsequently used for spread and direction estimation.

The selection process consists of three sequential steps:

1. Identification of valid candidate symbols within each region of the block.
2. Validation of the combination of candidates across regions.
3. Latching or updating of the selected symbols for further processing.

These steps are implemented by the *n-type slicer*, *cands_valid_combo*, and *cand_symbol_latch* modules, respectively.

Each region (first, middle, last) is processed by an independent *n-type slicer*. This module evaluates a small subset of symbols (three in this implementation) and determines whether a suitable candidate is present. A symbol is considered valid if it lies on the inner or outer ring of the constellation, which ensures reliable spread estimation (this criterion is further elaborated in the algorithmic description of the spread estimator). The index of the first valid symbol is returned via the *cand_idx* bus. If no valid candidate is found, the *cands_any* flag is deasserted.

The *cands_valid_combo* module evaluates the availability of candidates across the three regions. A valid combination is only accepted if at least two regions provide a candidate symbol. If this condition is not met, all candidate flags are suppressed. This constraint reduces the likelihood of unreliable spread or direction estimates, which would otherwise arise from insufficient spatial information within the block.

Finally, the *cand_symbol_latch* module selects and outputs the candidate symbols. For each region where the *cands_any* flag is asserted, the corresponding symbol is extracted from the input block using the provided *cand_idx*. If valid candidates are present, the outputs are updated and the *update* signal is asserted. Otherwise, the previous outputs are retained and the *hold* signal is asserted. In this way, consistency is maintained when no valid candidate combination is available.

### Timing flow

The timing behaviour of the input selector is illustrated in Figure 4.12.

The *n-type slicer* requires two clock cycles to determine and register a valid candidate index. The subsequent *cands_valid_combo* stage introduces an additional cycle to produce stable validity signals.

To ensure correct operation of the *cand_symbol_latch*, all inputs must be temporally aligned. In particular, the candidate indices and their corresponding validity flags must be synchronized with the original input symbols. To achieve this, delay elements are inserted in the data path. The input symbol block $\bar{r}$ is delayed by three clock cycles, matching the combined latency of the slicer and combination stages. The *cand_idx* signals are delayed by one cycle to align with the output of the *cands_valid_combo* module.

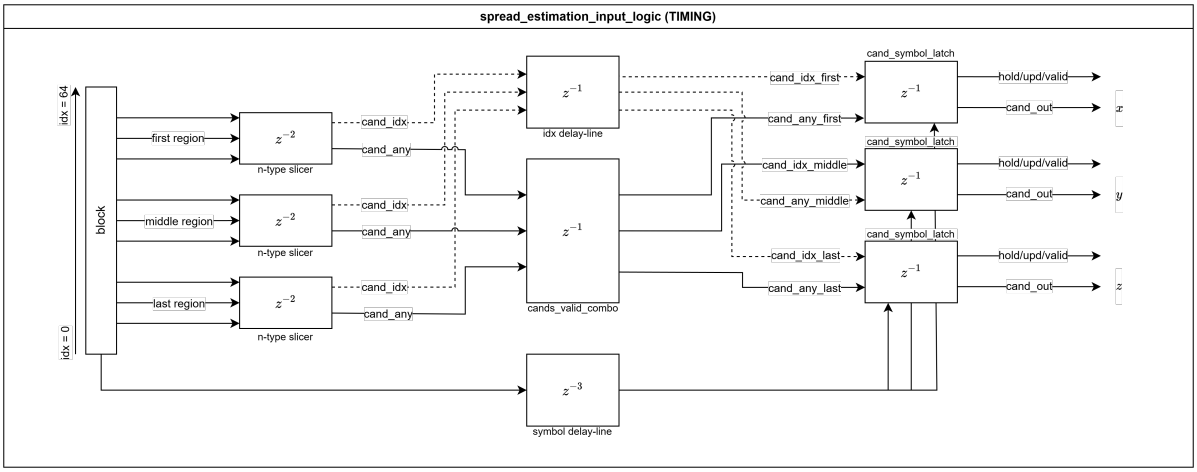



Figure 4.12: Timing flow diagram of the SE input selector

The *cand_symbol_latch* itself introduces one additional clock cycle to register the selected symbols. As a result, the total latency of the input selector amounts to four clock cycles.

### 4.4.2 Fourth-power I/Q calculators

#### Functional operation

The fourth-power calculators are an important part of the spread estimation algorithm. The purpose of the fourth-power operation is to suppress the rotational symmetry of the modulation and to make the carrier-induced phase evolution more directly observable.

For a complex input sample

$$r = I + jQ, \tag{4.6}$$

the block calculates

$$r^4 = (I + jQ)^4. \tag{4.7}$$

The output is again represented as a complex value, with a real and imaginary component. This operation is multiplication-heavy compared to most other operations in the low-complexity carrier recovery system. This is relevant because fourth-power based frequency or spread estimation is one of the computationally dominant operations in such systems: earlier work on this carrier recovery approach explicitly identifies the complex fourth-power operation as a large computation, requiring several real-valued multiplications per symbol [21].

In this implementation, the input samples are provided as signed fixed-point values in Q1.15 format. The fourth-power output is not rescaled back to Q1.15, but is kept at a much larger internal word length. This avoids unnecessary loss of precision in the following dot-product, sign and angle-estimation calculations without significantly effecting the compute needed for those calculations.

#### Data flow

Although the mathematical expression $r^4$ can be expanded directly, the hardware implementation avoids calculating all fourth-order terms independently. Instead, the calculation is structured as two consecutive squaring operations:

$$r^2 = (I + jQ)^2 = (I^2 - Q^2) + j(2IQ).$$

Defining

$$a = I^2 - Q^2, \qquad b = 2IQ,$$

the fourth power becomes

$$r^4 = (r^2)^2 = (a + jb)^2.$$

This gives

$$r^4 = (a^2 - b^2) + j(2ab). \tag{4.8}$$

The real and imaginary fourth-power outputs are therefore calculated as

$$\Re\{r^4\} = a^2 - b^2, \tag{4.9}$$

$$\Im\{r^4\} = 2ab. \tag{4.10}$$

This decomposition maps naturally onto a pipelined multiplier structure. The first stage calculates the three second-order products $I^2$, $Q^2$ and $IQ$. From these values, the second stage forms the intermediate real and imaginary components of $r^2$: $a = I^2 - Q^2$ and $b = 2IQ$. The multiplication by two is implemented as a left shift, so no additional multiplier is required. The third stage then calculates $a^2$, $b^2$ and $ab$. Finally, the fourth stage combines these terms into the real and imaginary components of the fourth-power output.

The resulting implementation requires six real multiplications: three multiplications in the first stage and three multiplications in the third stage. The remaining operations are additions, subtractions and shifts. This is more efficient than a direct polynomial expansion of $(I + jQ)^4$, because the intermediate square $r^2$ is reused.

The bit growth is also handled explicitly. Since the input samples are signed values with width $W$, the products $I^2$, $Q^2$ and $IQ$ require approximately $2W$ bits. The intermediate values $a$ and $b$ require one additional bit, because $a$ is formed by a subtraction and $b$ by a shift-left operation. Squaring $a$ and $b$ then doubles this width again. For this reason, the output width is chosen conservatively as

$$W_{\text{out}} \geq 4W + 3. \tag{4.11}$$

For the default input width of $W = 16$, this gives a required full-precision output width of at least 67 bits. The implementation uses a default output width of 70 bits, which provides additional margin and avoids overflow during the fourth-power calculation.

The use of a large fourth-power output width is acceptable here because the block is only applied to selected candidate symbols in the spread estimation path, rather than to every symbol in every blind phase search test angle. This fits the broader objective of the proposed architecture: reducing the number of expensive carrier recovery operations while preserving enough phase information to improve tracking of residual frequency offset.

### Timing flow

The fourth-power calculator is implemented as a four-stage registered pipeline. Figure 4.13 shows the calculation flow through these stages.

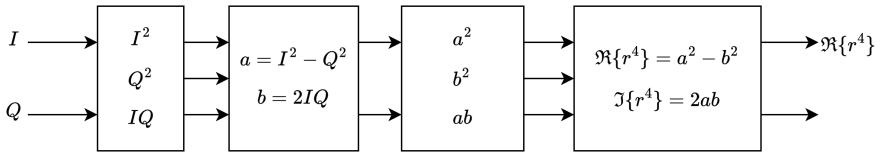



Figure 4.13: Calculation-flow of the fourth-power module

Each block in the figure corresponds to one clocked pipeline stage. The first stage performs the initial second-order products and registers the results internally. These intermediate values are then forwarded to the second stage, where the intermediate complex representation of the squared sample is formed.

The third stage contains the second multiplication layer of the architecture. Here, the intermediate terms generated in the previous stage are squared and multiplied again in preparation for the final fourth-power reconstruction. In the fourth and final stage, the real and imaginary outputs are assembled and driven to the output ports.

A valid signal is propagated alongside the datapath through the internal pipeline control signals `s1_valid`, `s2_valid`, `s3_valid` and `s4_valid`. This ensures that the output validity remains aligned with the corresponding fourth-power result throughout the pipeline.

The total latency of the module is therefore four clock cycles from assertion of `in_valid` to assertion of `out_valid`.

### CORDIC-based alternative

A possible alternative implementation would be to use a CORDIC-based polar approach. In such a design, the input sample would first be converted from Cartesian form to magnitude and phase:

$$r = |r|e^{j\phi}. \tag{4.12}$$

The fourth power could then be calculated as

$$r^4 = |r|^4 e^{j4\phi}. \tag{4.13}$$

Conceptually, this is attractive because the phase multiplication by four is simple, and the CORDIC algorithm is well suited for angle extraction and vector rotation.

However, this approach is not necessarily beneficial in the context of this low-complexity architecture. A CORDIC implementation would require an angle extraction step, additional handling of the magnitude term, and a rotation or sine/cosine reconstruction step. This introduces iterative logic, latency and control complexity. The fourth-power block used here, by contrast, is a feed-forward arithmetic pipeline consisting only of multipliers, adders, subtractors and shifts. It also produces exactly the Cartesian components required by the following dot-product and sign-estimation logic.

The main advantage of a CORDIC-based approach would be multiplier reduction, especially in an FPGA design where DSP slices are scarce or already heavily used. Its main disadvantage is that the algorithm becomes less direct: the system would temporarily move to a polar representation, only to return to Cartesian components for the following processing stages. For this design, the multiplier-based fourth-power calculator is therefore the considered implementation. It keeps the data path simple, deterministic and easy to pipeline, while avoiding the additional latency and format conversions of a CORDIC-based solution.

### 4.4.3 Spread estimation

This subsection describes the part of the spread-estimation backend that calculates the absolute phase-spread estimate. The focus here is the computation from a pair of selected complex samples to the unsigned angle estimate $|\theta|$.

#### Functional operation

After the selected samples have been raised to the fourth power, the spread-estimation block calculates the angle between the two resulting complex vectors. Let

$$r_0^{(4)} = I_0^{(4)} + jQ_0^{(4)}, \qquad r_1^{(4)} = I_1^{(4)} + jQ_1^{(4)}.$$

where $I_x(4)$ and $Q_x^{(4)}$ are the real and imaginary part respectively as also seen from equation 4.9. The numerator of the cosine estimate is the dot product of these two fourth-power vectors:

$$N = I_0^{(4)} I_1^{(4)} + Q_0^{(4)} Q_1^{(4)}.$$

This dot product is proportional to

$$|r_0^{(4)}|\,|r_1^{(4)}|\cos(4|\theta|),$$

so the required normalization factor is the reciprocal of the product of the two fourth-power vector magnitudes.

For the 16-QAM constellation used here, the denominator is not computed with a run-time divider. Instead, each sample is classified as belonging to the inner or outer diagonal radius of the constellation. The two inner/outer flags address a small radius-product LUT. This LUT directly returns a precomputed reciprocal multiplication factor for the possible radius products:

$$R_{\mathrm{II}}^{-1}, \qquad R_{\mathrm{IO}}^{-1}, \qquad R_{\mathrm{OO}}^{-1}.$$

The reciprocal factors are scaled fixed-point constants, so the normalization is implemented as one signed multiplication followed by an arithmetic right shift. The result is saturated to the interval representable by Q1.31 and represents

$$x = \cos(4|\theta|).$$

Add a small diagram showing the two parallel paths: fourth-power dot product and inner/outer reciprocal LUT, joining at the reciprocal multiplication.

The final conversion from $x$ to an angle is performed by an arccosine LUT. Since the desired output is

$$|\theta| = \frac{1}{4}\arccos(x),$$

the LUT stores $\arccos(x)/4$ rather than $\arccos(x)$. Only the half-domain $x \in [0, 1]$ is stored. For negative inputs, the symmetry

$$\frac{\arccos(-x)}{4} = \frac{\pi}{4} - \frac{\arccos(x)}{4}$$

is used to reconstruct the correct value. The output of this stage is therefore the absolute phase-spread estimate $|\theta|$, limited to the range $0 \le |\theta| \le \pi/4$.

#### Data flow

Add a plot of the stored LUT function $\arccos(x)/4$ for $x \in [0, 1]$, together with the symmetry reconstruction for $x < 0$.

The spread-estimation calculation is implemented as a fixed-point pipeline. The selected input samples are signed $G_{\mathrm{sample}}$-bit values. In the current implementation this corresponds to 16-bit signed I/Q samples. The fourth-power stage produces signed real and imaginary components with width $G_{\mathrm{pow4}}$. These components are kept at an extended precision because the subsequent dot product multiplies two fourth-power values.

The dot-product output has width $2G_{\mathrm{pow4}} + 1$ and is signed, since the angle between the two fourth-power vectors can produce a negative cosine. In parallel, the delayed original I/Q samples are converted into inner/outer radius flags. The radius-product LUT outputs a signed 48-bit reciprocal factor. The constants are scaled such that multiplying the dot product by the LUT value and shifting right by $G_{\mathrm{inv\_frac}}$ fractional bits produces a normalized Q1.31 value.

The normalized value is interpreted as a signed Q1.31 representation of $\cos(4|\theta|)$. Before addressing the arccosine LUT, the absolute value is taken. The special case $x = -1$ is saturated to the largest positive

Q1.31 value before taking the absolute value, avoiding overflow in two's-complement representation. The LUT address is formed from the upper fractional bits of $|x|$, specifically bits 30 down to 19, giving a 12-bit address and therefore 4096 table entries.

The arccosine LUT output is also Q1.31. It stores $\arccos(|x|)/4$ for non-negative input magnitudes. A delayed sign bit from the original Q1.31 cosine value selects either the direct LUT output or the reconstructed value $\pi/4 - \mathrm{LUT}(|x|)$. Thus the final output is a signed Q1.31 number, but it represents the non-negative angle $|\theta|$.

Add a fixed-point data-flow diagram with signal widths: input samples, fourth-power outputs, dot product, reciprocal LUT output, Q1.31 cosine, LUT address and Q1.31 angle.

**Timing flow**

The block is fully pipelined. Once the pipeline is filled, a new valid sample pair can be accepted every clock cycle, provided both input-valid signals are asserted. The latency from the selected input pair to the $|\theta|$ output is eight clock cycles in the current implementation.

Table 4.1: Pipeline latency of the absolute spread-estimation calculation.

| Block | Incremental latency | Cumulative latency |
|---|---|---|
| Fourth-power pipeline | 4 clk | 4 clk |
| Dot product | 1 clk | 5 clk |
| I/Q delay for denominator path | 3 clk | 3 clk |
| Inner/outer flag generation | 1 clk | 4 clk |
| Radius-product reciprocal LUT | 1 clk | 5 clk |
| Reciprocal multiplication and saturation | 1 clk | 6 clk |
| Arccos LUT address, BRAM read and sign alignment | 2 clk | 8 clk |

The fourth-power dot-product path and the denominator-reciprocal path are explicitly balanced. The denominator path starts with a three-clock delay of the original I/Q samples so that its reciprocal factor becomes valid in the same clock cycle as the dot product. The reciprocal multiplication stage only asserts its output-valid signal when both the dot product and the reciprocal factor are valid. The arccosine stage then adds two clocks, matching the LUT read latency and the delayed sign/valid signals used for half-domain reconstruction.

Add a timing diagram showing valid propagation through the dot-product path, reciprocal-denominator path, reciprocal multiplier and arccos LUT.

### 4.4.4 Sign estimation

**Functional operation**

sign pair decider go to first + middle or middle + last combination with that combo complex cross sign

**Data flow**

**Timing flow**

### 4.4.5 Filters

**Timing flow**

## 4.5 Phase correction block

## 4.6 Top-level integration

*This page is intentionally left blank.*

# 5

# Verification and simulation

You already have: python vectors testbenches error plots Put all here.

## 5.1 Simulation model

## 5.2 Test vector generation

## 5.3 VHDL testbench setup

## 5.4 Error analysis

### 5.4.1 Serialized blind phase search

### 5.4.2 Phase spread estimation

The absolute phase error between the floating-point model and the fixed-point implementation remains below ... rad for all tested blocks.

### 5.4.3 Phase unwrapper

### 5.4.4 Phase compensator

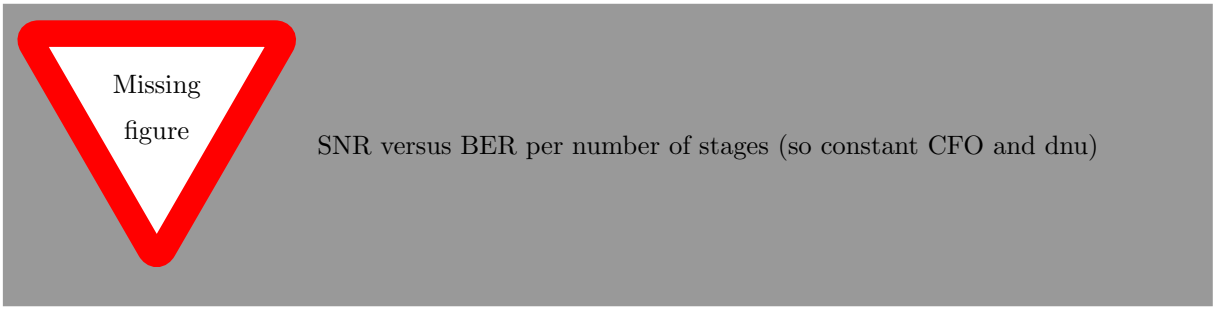
*This page is intentionally left blank.*

# 6

# Results

Possible results:
- phase error
- EVM
- resource usage
- timing
- block size effect

## 6.1 Serial blind phase search results




SNR versus BER per number of stages (so constant CFO and dnu)

## 6.2 Phase spread estimation results

## 6.3 Error versus SNR

## 6.4 Resource usage

## 6.5 Latency

## 6.6 Comparison with reference method

*This page is intentionally left blank.*

# 7

# Discussion

## 7.1 Limitations

## 7.2 Possible improvements

## 7.3 Design trade-offs

*This page is intentionally left blank.*

# 8

# Conclusion

*This page is intentionally left blank.*

# Bibliography


[1] Coherent Corp., "Coherent to demonstrate next-generation transceiver and semiconductor laser technology for 800g and 1.6t transmission in ai networks at ecoc 2023." `https://www.coherent.com/news/press-releases/coherent-to-demo-next-gen-transceiver-and-semiconductor-tech-at-ecoc`, Oct. 2023. accessed April 9, 2026.

[2] Jingchi Cheng, et al., "Comparison of coherent and imdd transceivers for intra datacenter optical interconnects." `https://www.researchgate.net/publication/331337485_Comparison_of_Coherent_and_IMDD_Transceivers_for_Intra_Datacenter_Optical_Interconnects`, Jan. 2019. accessed April 9, 2026.

[3] M. S. Wartak, *Computational Photonics: An Introduction with MATLAB*. Cambridge University Press, 2013. Chapter 10: Optical receivers.

[4] J. M. Kahn and K.-P. Ho, "Spectral efficiency limits and modulation/detection techniques for dwdm systems," *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 10, no. 2, pp. 259–272, 2004.

[5] S. J. Savory, "Digital coherent optical receivers: Algorithms and subsystems," *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 16, no. 5, pp. 1164–1179, 2010.

[6] M. Kuschnerov, F. N. Hauske, K. Piyawanno, B. Spinnler, M. S. Alfiad, A. Napoli, and B. Lankl, "Dsp for coherent single-carrier receivers," *Journal of Lightwave Technology*, vol. 27, no. 16, pp. 3614–3622, 2009.

[7] X. Zhou, R. Urata, and H. Liu, "Beyond 1 tb/s intra-data center interconnect technology: Im-dd or coherent?," *Journal of Lightwave Technology*, 2020.

[8] K. Kikuchi, "Fundamentals of coherent optical fiber communications," *Journal of Lightwave Technology*, 2016.

[9] E. Ip and J. M. Kahn, "Feedforward carrier recovery for coherent optical communications," *Journal of Lightwave Technology*, vol. 25, no. 9, pp. 2675–2692, 2007.

[10] T. Pfau, S. Hoffmann, and R. Noé, "Hardware-efficient coherent digital receiver concept with feedforward carrier recovery for m-qam constellations," *Journal of Lightwave Technology*, vol. 27, no. 8, pp. 989–999, 2009.

[11] I. Roudas, "Coherent optical communication systems," in *Optical Networks: WDM Systems and Networks*, Springer, 2011.

[12] A. Leven, N. Kaneda, U.-V. Koc, and Y.-K. Chen, "Frequency estimation in intradyne reception," *IEEE Photonics Technology Letters*, vol. 19, no. 6, pp. 366–368, 2007.

[13] M. Selmi, Y. Jaouen, and P. Ciblat, "Accurate digital frequency offset estimator for coherent polmux qam transmission systems," in *Proceedings of the European Conference on Optical Communication (ECOC)*, 2009. Paper P3.08, Vienna.

[14] J. C. Rasmussen, "Advances in coherent detection algorithms," in *Proceedings of SPIE*, vol. 7960, 2011.

[15] A. Devices, "Mixed-signal and dsp design techniques: The dft/fft (bin spacing and frequency resolution)." PDF technical handbook chapter.

[16] H.-P. . A. Technologies, "Spectrum and signal analyzer measurements and noise." Technical measurement guide (FFT/bin-based resolution and estimation effects).

[17] T. Yang *et al.*, "Linewidth-tolerant and multi-format carrier phase estimation based on blind phase search," *Optics Express*, 2018.

[18] J. E. Volder, "The cordic trigonometric computing technique," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.

[19] P. K. Meher, J. Valls, T.-B. Juang, K. Sridharan, and K. Maharatna, "50 years of cordic: Algorithms, architectures, and applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 56, no. 9, pp. 1893–1907, 2009.

[20] A. (Xilinx), "Cordic v6.0 logicore ip product guide (pg105)," 2021. Release date 2021-08-06.

[21] e. a. Jonas De Busscher, Axl Bomhals, "Low complexity carrier recovery system design for baudrate sampled coherent datacenter interconnects," *IEEE Communications Letters*, 2025.

[22] D. A. A. Mello, F. A. Barbosa, and J. D. Reis, "Interplay of probabilistic shaping and the blind phase search algorithm," *Journal of Lightwave Technology*, vol. 36, p. 5096–5105, Nov. 2018.

*This page is intentionally left blank.*

*This page is intentionally left blank.*